Vietnamese-German University

Department of Electrical Engineering & Information Technology

Frankfurt University of Applied Science

Faculty 2: Computer Science and Engineering

Design of an ESP32 Firmware for a Battery - Powered Wireless Sensor Network

Duy Dang Vu Minh

Matriculation number: 1317899

Supervisor: Dr. Chan Thai Truyen Dai

Advisor: M.Sc. Thuyet Nguyen Vo That

BACHELOR THESIS REPORT

Submitted in partial fulfillment of the requirements for the degree of Bachelor Engineering in study program Electrical and Computer Engineering, Vietnamese-German University

Ho Chi Minh City, Vietnam, March 2026

VIETNAMESE – GERMAN UNIVERSITY
ELECTRICAL & COMPUTER ENGINEERING STUDY PROGRAM

BACHELOR THESIS REPORT

Design of an ESP32 Firmware for a Battery - Powered Wireless Sensor Network

Author:

Duy Dang Vu Minh

Approved by:

Dr. Chan Thai Truyen Dai

M.Sc. Thuyet Nguyen Vo That

Ho Chi Minh City, Vietnam, March 2026

Declaration

I hereby declare that this thesis entitled “Design of an ESP32 Firmware for a Battery - Powered Wireless Sensor Network” is submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering.

This project was carried out as a team project consisting of three members responsible for hardware, firmware, and web application development, respectively.

The work presented in this thesis represents the author’s individual contribution to the project, specifically the design, implementation, and evaluation of the ESP32-S3 firmware architecture and the associated cloud ingestion pipeline.

All other contributions from team members have been acknowledged where appropriate.

I confirm that the content of this thesis is my own work and has not been submitted for any other degree or qualification.

I also declare that all opinions, results, conclusions, and recommendations are my own and may not represent the policies or opinions of Electrical and Computer Engineering, study program of the Vietnamese German University.

Signature:

Name: Dang Vu Minh Duy

Student ID: 13885

Matriculation number: 1317899

Date: 26th February 2026

Abstract

This thesis presents the design and evaluation of a low-power firmware for the ESP32-S3 microcontroller and a cloud system for collecting data, specifically made for battery-operated wireless sensor nodes. The system solves important problems in wireless sensor networks, such as strict energy limits, unreliable connections, data reliability, and precise time synchronization.

On the device side, the firmware utilizes a duty-cycled architecture with deep sleep scheduling and periodic wake-ups. Sensor data are buffered locally using non-volatile storage solutions (ESP32 Preferences and LittleFS), then transmitted in batches to minimize communication overhead. Two data transmission approaches are assessed: Wi-Fi-based HTTP and cellular-based HTTPS via a cellular modem. A robust time recovery mechanism—combining Network Time Protocol (NTP) and HTTP header analysis ensures timestamp accuracy even after power interruptions.

The cloud side uses a serverless system built with Firebase Cloud Functions that handles incoming data, checks records, calculates statistics for different time periods (5 minutes, hourly, and daily), and generates alerts based on predefined limits.

Experimental results demonstrate that batch transmission significantly reduces network overhead while preserving elevated data reliability amidst intermittent connectivity. The proposed architecture offers an energy-efficient and fault-tolerant solution well-suited for distributed environmental monitoring applications.

Keywords: ESP32-S3, Wi-Fi and LTE communication, deep sleep, store-and-forward buffering, Firebase Cloud Functions.

Acknowledgements

I would like to express my sincere gratitude to my supervisor for the valuable guidance, constructive feedback, and continuous support throughout the completion of this thesis.

First and foremost, I would like to sincerely thank Dr. Chan Thai Truyen Dai and M.Sc. Thuyet Nguyen Vo That for their invaluable guidance, encouragement, and constructive feedback throughout this project. Their expertise and continuous support have played a crucial role in shaping the direction and quality of this work.

I am also deeply grateful to my team members, Diem Pham Hien and Duc Pham Nguyen Minh, for their valuable insights, thoughtful feedback, and encouragement.

This project was conducted as a collaborative effort involving hardware, firmware, and web application development. I would like to thank my teammates for their cooperation and contributions to the overall system integration and development.

Finally, I would like to express my heartfelt appreciation to my family for their unwavering support, patience, and encouragement throughout this journey. Their constant motivation has given me the strength to complete this thesis.

Duy Dang Vu Minh

Table of Contents

Declaration ii
Abstract iii
Acknowledgements iv
Table of Contents v
List of Figure x
List of Table x
Acronyms / Abbreviations xi
1. Introduction 1
1.1 Background 1
1.2 Problem Statement 2
1.2.1 Energy Constraints Under Variable Harvesting Conditions 2
1.2.2 Intermittent and unreliable network connectivity 2
1.2.3 Timestamp Integrity Across Deep Sleep Cycles and Power Interruptions 3
1.2.4 Backend scalability and actionable processing 3
1.3 Research Objectives 3
1.4 Scope of work 4
1.5 Contribution 5
1.6 Thesis Organization 6
2. Literature Review 7
2.1 Wireless Sensor Networks for Environmental Monitoring 7
2.2 Battery-Powered and Energy-Constrained IoT Systems 8
2.3 Local Buffering and Store-and-Forward Mechanisms in Wireless Sensor Networks 9
2.4 Time Synchronization in Distributed Sensor Networks 10
2.5 Cloud-Based IoT Data Processing and Aggregation Architectures 11
2.6 Research Gap Summary 12
3. Embedded System Architecture 13
3.1 System Architecture Overview 13
3.2 Embedded Platform and Sensor Emulation Interfaces 15
3.2.1 Analog ADC-Based Emulation 15

3.2.2 Modbus RTU Over RS485 15
3.2.3 Real Time Clock Over I2C Protocol 16
3.3 Firmware Architecture 18
3.3.1 Operating Cycle 18
3.3.2 Data Acquisition Layer 19
3.3.3 Local Buffering Mechanism 19
3.4 Data Flow and Communication Strategy 21
3.5 Power Management Strategy 23
4. Firmware Design and Implementation 25
4.1 Firmware Design Requirements 25
4.2 Implementation of Operating Cycle 27
4.3 Implementation of Data Acquisition Logic 29
4.3.1 Analog ADC-Based Input (Development Support) 29
4.3.2 Modbus RTU Measurement Retrieval (Primary Interface) 29
4.3.3 I2C Interface Input 30
4.3.4 Output Normalization and Record Preparation 30
4.3.5 Development Iteration: Initial Modbus TCP Testing (Prototype Development) 30
4.4 Local Data Buffering Implementation 32
4.4.1 Preferences-Based Queue Mechanism (Primary Implementation) 32
4.4.2 Batch Transmission and Queue Clearing Logic 32
4.4.3 LittleFS-Based Buffering (Prototype Development) 33
4.5 Communication and Cloud Transmission Implementation 35
4.5.1 Network Initialization and Connection Handling 35
4.5.2 Batch Payload Construction 35
4.5.3 Wi-Fi Upload via HTTPS 36
4.5.4 Cellular Upload via SIMCOM A7682S 36
4.5.5 TCP Modbus Implementation (Prototype Development) 36

4.5.6 Energy Considerations During Communication 37
4.6 Time Synchronization and Timestamp Handling 39
4.6.1 Initial Time Synchronization 39
4.6.2 Timestamp Progression Across Deep Sleep 39
4.6.3 Handling Communication or Synchronization Failure 40
4.6.4 Timestamp Integration in Batch Payload 40
4.7 Firmware Reliability and Error Handling 42
4.7.1 Network Connection Failure Handling 42
4.7.2 Transmission Failure Handling 42
4.7.3 Sensor Interface Failure Handling 42
4.7.4 Safe Deep Sleep Recovery 43
4.8 Firmware Summary 43
5. Cloud Backend Processing Design 44
5.1 Cloud System Architecture 44
5.2 Realtime Database Processing Pipeline 46
5.2.1 Data Upload from ESP32 to Realtime Database 46
5.2.2 Event-Triggered Bridge Function 46
5.2.3 Firestore Structured Storage 47
5.2.4 Multi-Interval Statistical Aggregation 47
5.3 Threshold Evaluation Logic 49
5.4 Database Organization 50
5.5 Development Utilities (Migration Function) 50
5.6 Direct HTTP Ingestion Pipeline (Prototype Development) 51
5.7 Cloud Backend Summary 51
6. Experimental Evaluation 52
6.1 Experimental Setup 52
6.2 Operating Cycle and Timing Analysis 54
6.3 Overhead Communication Evaluation 56
6.4 Data Reliability Under Network Interruption 58

6.5 Timestamp Consistency Evaluation 60
6.6 Backend Aggregation Verification 62
7. Conclusion and Future work 63
7.1 Conclusion 63
7.2 Limitations 64
7.3 Future Work 64
References 65
Appendices 67
Appendix A: Source Code Repository 67
A.1 Repository link 67
A.2 Repository structure 67
A.3 File list and purpose 67
Appendix B: Firmware Overview and Configuration 68
B.1 Common operating cycle 68
B.2 Pin mapping (main variants) 68
B.3 Data acquisition 69
B.4 Local buffering format (Wi-Fi and SIMCOM) 69
B.5 Wi-Fi upload method 70
B.6 SIMCOM upload method 70
B.7 Wi-Fi TCP (Prototype) 70
Appendix C: Sensor Emulation and Test Setup 71
C.1 Potentiometer-based emulation (Wi-Fi firmware) 71
C.2 Modbus RTU emulation with pyModSlave (SIMCOM firmware) 71
C.3 Modbus TCP emulation with pyModSlave (prototype firmware) 71
Appendix D: Cloud Backend Functions and Data Model 72
D.1 Location 72
D.2 Two ingestion paths in the Cloud Function 72
D.3 RTDB record format (main path) 72

D.4 Firestore structure (main path) 73

D.5 Status classification and alert codes 73

D.6 Aggregation windows 73

D.7 Migration tool 74

Appendix E: Sample JSON Payloads and Database Structure 74

E.1 Example Wi-Fi/SIMCOM payload written to RTDB (PATCH body) 74

E.2 Example prototype request body (HTTP ingestBatch path) 75

E.3 Example Firestore readings/{id} document (main path) 75

E.4 Example Firestore aggregation document (stats buckets) 76

List of Figures

Figure 3.1 System Architecture Overview 14
Figure 3.2 Embedded Platform and Sensor Emulation Interfaces 17
Figure 3.3 Firmware Operational Flow 20
Figure 3.4 Data Transmission Strategy 22
Figure 3.5 Duty-Cycle Based Power Operation 24
Figure 4.1 Firmware Design Overview 26
Figure 4.2 Wake-up Execution Cycle 28
Figure 4.3 Data Acquisition Pathways (ADC vs Modbus RTU) 31
Figure 4.4 Store-and-Forward Buffering Workflow 34
Figure 4.5 Communication and Transmission Sequence 38
Figure 4.6 Timestamp Handling Across Deep Sleep Cycles 41
Figure 5.1 Cloud Backend Architecture 45
Figure 5.2 Statistical Aggregation 48
Figure 6.1 Experimental Setup 53
Figure 6.2 Operating Cycle Timing 55
Figure 6.3.1 Estimated battery lifetime (batch upload per 5-minute) 56
Figure 6.3.2 Estimated battery lifetime (single upload per measurement) 57
Figure 6.4.1 Device upload stored measurements after network connectivity was restored 58
Figure 6.4.2 Device keeps batch measurements after network interrupts 59
Figure 6.5.1 Sequential timestamps across multiple cycles (cloud storage) 60
Figure 6.5.2 Sequential timestamps across multiple cycles (local storage) 61
Figure 6.6 Backend Aggregation 62

List of Tables

Table 5.3 Thresholds range 49
Table B.2 Pin mapping of the main firmware variants 69

Acronyms / Abbreviations

ADC	Analog-to-Digital Converter
APN	Access Point Name
CPU	Central Processing Unit
DS3231	Real-Time Clock (RTC) Module DS3231
ESP32-S3	Espressif ESP32-S3 Microcontroller
FS	File System
GPRS	General Packet Radio Service
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
I2C	Inter-Integrated Circuit
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
LAN	Local Area Network
LED	Light-Emitting Diode
LTE	Long-Term Evolution (4G Cellular Communication)
mAh	Milliampere-hour
Modbus	Industrial Communication Protocol
NTP	Network Time Protocol
NVS	Non-Volatile Storage (ESP32 Preferences Library)
PATCH	HTTP Method for Partial Update
REST	Representational State Transfer
RS-485	Recommended Standard 485 (balanced serial communication)
RSSI	Received Signal Strength Indicator
RTC	Real-Time Clock
RTDB	Firebase Realtime Database

SPIFFS	Serial Peripheral Interface Flash File System
LittleFS	Lightweight Flash File System
SSL	Secure Sockets Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
UTC	Coordinated Universal Time
WSN	Wireless Sensor Network

1. Introduction

1.1 Background

Monitoring water salinity is essential for environmental management, aquaculture, and agricultural irrigation systems [1]. In aquaculture farms and coastal ecosystems, maintaining suitable salinity range is crucial for the health and growth of aquatic species. Sudden variations in salinity can result in significant economic losses or ecological disruption. Continuous and real-time monitoring offers substantial benefits compared to conventional manual sample techniques [2].

Advancements in Internet of Things (IoT) technologies have made Wireless Sensor Networks (WSNs) a viable alternative for automated environmental monitoring [3]. In this context, a WSN refers to a set of distributed sensor nodes that periodically measure environmental parameters and deliver them to a central storage/processing system. Low-cost microcontroller platforms such as the ESP32-S3 enable sensing, local data handling, and wireless communication within compact and economical devices. When integrated with cloud-based data storage and analytics, such solutions provide remote monitoring, alert generation and long-term trend analysis [4].

Salinity monitoring stations are frequently situated in distant ponds, coastal regions, or agricultural locations without wired power infrastructure. To facilitate independent operation, these devices are generally powered by rechargeable batteries supported by solar energy harvesting [5]. Although solar panels generate sustainable energy, the output varies based on environmental factors such as sunshine intensity and weather conditions. Therefore, the firmware must be carefully designed to reduce energy usage and remain reliably under constrained or fluctuating power conditions.

Wireless communication, particularly through Wi-Fi or cellular network, incurs considerable power overhead from connection establishment, data transmission, and protocol negotiation [6]. Excessive frequency in measurement transmission may result in battery depletion during periods of low solar input. Conversely, delaying transmissions without a robust buffering strategy can cause data backlog growth and potential data loss.

Moreover, accurate timestamping is crucial for environmental data processing and large-scale cloud gathering. Power outages or system reboots may compromise real-time clock values, leading to inconsistent historical data. Consequently, dependable time synchronization techniques are essential to maintain temporal data integrity [7].

Beyond data upload, practical monitoring systems also require backend processing such as data validation, threshold-based status evaluation, and multi-resolution aggregation (e.g., 5-minute, hourly, and daily summaries) to reduce storage overhead and support actionable visualization and decision-making [4]. These requirements motivate an end-to-end design where firmware behavior (sampling, buffering, batching, and timestamping) is aligned with backend ingestion and aggregation logic.

This thesis presents the design and evaluation of a battery-operated (solar-charged) salinity monitoring architecture based on the ESP32-S3 microcontroller, emphasizing low-power firmware design and cloud-based data processing logic. During development, salinity and pH streams are emulated to validate firmware and backend behavior prior to full sensor integration.

1.2 Problem Statement

Battery-operated (solar charged) wireless sensor nodes are a beneficial way to monitor the environment from a distance. However, practical deployment introduces several technical challenges that must be addressed to ensure reliable long-term operation:

1.2.1 Energy Constraints Under Variable Harvesting Conditions

Solar energy availability strongly depends on environmental conditions such as sunlight intensity and weather variability [5]. During periods of low sunlight intensity, available battery energy may decrease significantly. So, if not properly controlled, constant wireless connectivity, especially Wi-Fi reconnection or cellular network attachment, will quickly drain stored energy. Immediate transmission of every measurement may compromise long-term system autonomy.

1.2.2 Intermittent and unreliable network connectivity

Not only are there energy constraints, but network connectivity is also often unstable in remote water monitoring regions. If Wi-Fi or cellular service goes down for a short time, it may

not be possible to send data in real time. If there isn't a good way to buffer data locally, sensor readings could be lost forever, making environmental monitoring less reliable [3]. A store-and-forward approach is needed so data can be preserved locally and transmitted once connectivity is restored.

1.2.3 Timestamp Integrity Across Deep Sleep Cycles and Power Interruptions

Another big problem is making sure that timestamps are accurate. When the battery voltage goes below safe levels, Battery-powered (solar charged) systems may lose power or must be reset. In some situations, real-time clock values may not be right, which might lead to wrong timestamps and cloud records that don't match. Accurate long-term data analysis requires reliable timestamp recovery and synchronization mechanisms [7].

1.2.4 Backend scalability and actionable processing

Storing raw sensor measurements in the cloud isn't enough for monitoring apps that work in real life. A scalable backend architecture must provide data validation, alert creation based on threshold evaluation, and multi-resolution statistical aggregation (e.g., 5-minute, hourly, and daily summaries) to make it easier to see and make decisions [4].

These challenges highlight the need for an integrated energy-aware firmware and cloud architecture that ensures efficient operation, reliable data delivery, accurate timestamp management, and scalable backend processing.

1.3 Research Objectives

The main objective of this thesis is to design and evaluate a battery-powered (solar charged) salinity monitoring system based on the ESP32-S3 platform, focusing on energy efficiency, data reliability, and scalable cloud integration.

To achieve this goal, the specific objectives are separated as follows:

1. To develop a low-power firmware architecture for the ESP32-S3 sensor node utilizing deep sleep scheduling and duty-cycled operation to reduce energy usage within battery-powered based on solar charged limitations.

2. To implement a dependable offline data buffering system utilizing non-volatile storage (Preferences and LittleFS) to avert data loss during network interruptions and guarantee store-and-forward transmission.
3. To design a batch-based data transmission strategy that reduces communication overhead by collecting various sensor readings before uploading via Wi-Fi or cellular connectivity.
4. To implement accurate time synchronization and recovery mechanisms, such as NTP-based synchronization and HTTP header-based time extraction, to maintain timestamp integrity during power interruptions.
5. To build a serverless cloud ingestion system that can validate incoming data, produce threshold-based warnings, and compute multi-time-period statistical aggregates (5-minute, hourly, and daily summaries).
6. To experimentally evaluate the system performance, including communication overhead, reliability under interrupt connectivity, and data consistency within the cloud platform.

1.4 Scope of work

This thesis is limited to the firmware development and selected cloud backend components of the proposed battery-powered salinity monitoring system. During the development phase, salinity, pH and temperature data were simulated to validate firmware functionality prior to full hardware sensor integration.

On the embedded side, the work includes the design and implementation of the ESP32-S3 firmware, which is responsible for sensor data acquisition, energy-efficient operation via deep sleep scheduling, local data buffering using NVS mechanisms, and batch-based wireless transfer via Wi-Fi and cellular connectivity. The firmware also includes time synchronization mechanisms to maintain timestamp integrity after power interruptions in a battery-powered environment.

On the cloud side, this thesis focuses specifically on the implementation of data ingestion and processing logic. This includes developing HTTP-based batch ingestion functions, data validation routines, threshold-based alert evaluation, and a range of statistical aggregates (5-

minute, hourly, and daily summaries) using serverless architecture. Migration and bridging mechanisms between Realtime Database and Firestore are also within the scope of this work.

The entire cloud infrastructure setup, user interface development, and hardware design of the system are beyond the scope of this thesis and were handled collaboratively by the project members.

1.5 Contribution

The primary contributions of this thesis are summarized as follows:

1. **Design of an energy-efficient firmware architecture for battery-powered operation.**
   The firmware integrates deep sleep scheduling, duty-cycled operation and data handling, and batch-based transmission strategies to minimize communication overhead and extend battery lifetime under variable solar energy conditions.
2. **Implementation of a store-and-forward buffering mechanism.**
   Two NVS mechanisms (Preferences and LittleFS) were implemented to ensure data permanence during network disruptions. To avoid data loss and redundant transmission, a controlled backlog management and compaction technique was implemented.
3. **Development of accurate time synchronization and recovery mechanisms.**
   The system incorporates both NTP-based synchronization and HTTP header-based time extraction methods to maintain timestamp integrity after power interruptions, ensuring consistent cloud-side aggregation.
4. **Design and implementation of cloud-based ingestion and aggregation logic.**
   A serverless backend was implemented to process batch ingestion, data validation, threshold-based alert generation, and automatic statistical aggregation at multiple periods (5-minute, hourly, and daily levels).
5. **End-to-end integration of energy-aware embedded firmware and cloud data processing architecture.**

The work shows a comprehensive data flow from low-power embedded firmware to structured cloud storage and multi-period statistical aggregation under intermittent connectivity conditions.

1.6 Thesis Organization

The following parts of this thesis are structured as follows:

Chapter 2 discusses relevant research on wireless sensor networks, battery-powered (solar charged) Internet of Things devices, low-power firmware development, and cloud-based environmental monitoring frameworks.

Chapter 3 describes the overall system architecture, including the interaction among the embedded node, wireless communication modules, and the cloud backend.

Chapter 4 defines the design and execution of the ESP32-S3 firmware, encompassing power management, deep sleep scheduling, local buffering techniques, time synchronization, and batch-oriented wireless transmission.

Chapter 5 clarifies the cloud backend logic, encompassing data intake, validation, alarm assessment, and multi-resolution statistical aggregate.

Chapter 6 evaluates the system's performance in the scenario of interrupted connection and solar power limitations.

Chapter 7 concludes the thesis and outlines future development directions.

2. Literature Review

2.1 Wireless Sensor Networks for Environmental Monitoring

Wireless Sensor Networks (WSNs) have been widely adopted in environmental monitoring applications due to their ability to collect data from various locations and transmit it wirelessly to a central system [2] [3]. A standard WSN consists of sensor nodes that have sensing components, a microprocessor, communication modules, and a power supply. These nodes operate collaboratively to monitor physical or environmental conditions such as temperature, humidity, or water-related metrics.

Several studies have discussed the architecture and design challenges of WSNs. Akyildiz et al. [2] pointed out that energy efficiency, scalability, and reliable communication are key constraints in practical deployments. Since most sensor nodes operate on limited power sources, energy management has become a crucial component of architecture. Yick et al. [3] further analyzed communication protocols and fault tolerance strategies, emphasizing that unstable connectivity and node failures are common in real-world environments.

In water monitoring applications, WSN-based systems have been proposed to measure parameters such as pH, salinity, and temperature [8] [9]. These devices prove that automated environmental data collecting is possible, in comparison to manual sampling. However, many implementations assume either stable power conditions or short-term operation periods. In outside aquaculture or agricultural environments, those assumptions may not always hold.

With the growth of IoT platforms, cloud integration has become increasingly popular in environmental monitoring systems [10]. Cloud-based architecture allows long-term data storage, remote visualization, and automated alert mechanisms. Nevertheless, in many reported systems, the embedded node and the cloud backend are seen as independent parts instead of as a tightly integrated architecture. Energy-aware firmware design and cloud-side aggregation logic are often processed separately.

In general, despite WSN technology for environmental monitoring being well established, there still exists a need for more integrated methods that include energy limitations, discontinuous connections, and scalable cloud processing within a unified system architecture.

2.2 Battery-Powered and Energy-Constrained IoT Systems

Many environmental monitoring systems have been deployed in places where there isn't any connected power infrastructure. In these situations, solar energy harvesting and rechargeable batteries are frequently used together to maintain devices running for an extended period [11] [12].

In contrast to normal energy sources, solar energy is highly dependent on weather patterns, seasonal variations, and the intensity of sunlight. During periods of limited sunlight, the battery may not be charged enough, which would limit the amount of energy that can be used for tasks like sensing, processing, and communication. In their discussion of the idea of energy-neutral operation in energy-harvesting sensor networks, Kansal et al [11], emphasized how crucial it is to balance energy consumption and harvested energy in order to prevent system shutdown.

In practical IoT nodes, wireless communication is generally the most energy-consuming activity. Research has demonstrated that compared to local sensing or processing, connection establishment, protocol negotiation, and data transmission consume significantly more power [13]. This is particularly true for Wi-Fi and cellular-based systems, where network connection procedures and data transfer protocols incur considerable overhead. Therefore, frequent real-time transmission may rapidly drain battery reserves, especially in solar battery-powered deployments where energy input varies throughout the day

To overcome these challenges, several energy-aware design techniques have been proposed. Some common methods are duty cycling, deep sleep scheduling, adaptive sampling rates, and transmission batching [12] [14]. Duty cycling minimizes active operating time by placing the microcontroller into low-power mode when sensing and communication are not required. Batch transmission strategies reduce communication overhead by aggregating multiple measurements before sending them to the cloud, thereby decreasing the number of times network connections are made.

While existing studies provide helpful information about energy harvesting models and low-power hardware techniques, fewer works emphasize the integration of energy-aware firmware behavior with higher-level data processing strategies. In real-world deployments, energy efficiency should not be treated solely as a hardware problem. Instead, it must also impact firmware scheduling decisions, buffering mechanisms, and communication processes.

In order to maintain reliable long-term operation under fluctuating energy and connectivity conditions, the design of solar-powered monitoring systems necessitates a coordinated approach that incorporates energy-aware firmware architecture, controlled communication strategies, and backend processing considerations.

2.3 Local Buffering and Store-and-Forward Mechanisms in Wireless Sensor Networks

Reliable data delivery is an important requirement in environmental monitoring systems, especially when devices are deployed in remote areas. In practice, wireless communication is not always stable. Signal loss, interference, temporary infrastructure failures, or energy-related shutdowns may interrupt data transmission. If measurements are sent immediately without any local storage mechanism, temporary network failure may lead to permanent data loss.

To minimize this risk, numerous wireless sensor network architectures implement a store-and-forward methodology [15] [16]. In this method, collected data is stored locally when transmission is not possible. Once connectivity is restored, the stored data is transmitted to the central server. This mechanism improves system robustness and ensures data continuity.

Local buffering generally utilizes non-volatile storage memory to preserve unsent records, even during power interruptions. IoT systems occasionally utilize flash memory, EEPROM, or file-based storage systems to store measurements that have not been received [17]. Flash-based storage is commonly used due to its availability in embedded systems. However, storage capacity and memory endurance must be considered. Excessive write operations may reduce flash lifetime, and insufficient buffer size may lead to data overflow. Therefore, to control memory used and avoid overflow, the systems commonly choose efficient backlog management strategies such as compaction, head-pointer tracking, or circular buffer architectures [17] [18].

Another commonly used strategy is batch transmission [19]. Instead of sending every reading immediately, multiple measurements are grouped into a single transmission. This method minimizes communication overhead and decreases the number of network reconnections. In energy-constrained systems, a reduction in transmission events typically results in lower overall power consumption.

Despite extensive discourse in the literature, buffering and batch transmission are occasionally considered as optional features rather than core architectural components. In long-term monitoring systems operating under intermittent connectivity and limited energy availability, these mechanisms become essential rather than additional.

2.4 Time Synchronization in Distributed Sensor Networks

In environmental monitoring systems, timestamps play an important role in processing data. Measurements collected over long periods must be linked to time information to support trend analysis and statistical aggregation. Invalid timestamps may lead to inaccurate results in cloud-level data processing.

In distributed wireless sensor networks, maintaining synchronized time across nodes presents several challenges. The internal clocks of microcontrollers may drift over time, especially when operating under temperature variations or unstable power conditions. Even tiny clock deviations can accumulate and affect the accuracy of recorded data [7].

A popular method for time synchronization is the Network Time Protocol (NTP) [20]. Embedded devices can periodically synchronize their local clocks by connecting to an external time server. While this method is widely adopted in IoT systems, it requires system active network connectivity. In energy-constrained systems, repeated synchronization attempts may increase communication overhead.

Several systems deploy lightweight alternative methods. For instance, temporal data can be extracted from HTTP response headers during regular data uploads [21]. The device reuses the timestamp from the server response instead of initiating a separate synchronization request. This mechanism reduces additional communication overhead and may be more suitable for low-power deployments.

Power interruptions cause another practical issue. In solar-powered systems, insufficient battery voltage may cause reboots of the whole system. If the internal clock is not preserved, timestamps may become invalid after reboots. To maintain accuracy, embedded systems often integrate external real-time clock (RTC) module with periodic network-based synchronization [22].

Although time synchronization is often considered a standard system feature, its reliability directly affects data integrity in long-term environmental monitoring applications. For systems operating under intermittent connectivity and variable energy availability, time recovery and correction mechanisms must be carefully designed.

2.5 Cloud-Based IoT Data Processing and Aggregation Architectures

With the growth of IoT platforms, cloud integration has become an essential component in environmental monitoring systems. Instead of storing data locally for short periods, modern systems transmit measurements to cloud servers for long-term storage and processing. Cloud-based systems allow remote access, centralized analytics, and automated alert generation.

Most IoT systems depend on standard communication protocols such as HTTP or MQTT to transmit data from embedded nodes to backend-cloud servers [23] [24]. After receiving it, the data are authenticated and archived in structured databases. Cloud platforms also enable scalability, allowing systems to manage increasing numbers of devices without major architectural changes.

Besides storing raw measurements, cloud systems frequently execute additional processing. These might include threshold-based warning detection, raw measurements anomaly recognition, and statistical aggregation across various time intervals. For instance, instead of storing only raw measurements every single reading at full period, several systems generate hourly or daily summaries to minimize data volume and scale long-term trend analysis [4].

Another common method widely applied in IoT systems is batch-based ingestion. Instead of processing an individual record separately, the server processes multiple records in a single request. This method reduces server overhead and improves efficiency when handling large volumes of data [25]. Batch ingestion is especially advantageous when devices operate under discontinuation connections and upload buffered data after reconnection.

Overall, cloud-based processing has become an essential part of modern environmental monitoring systems. However, firmware design and cloud-backend processing are often developed independently. In practical deployments, the behavior of the embedded node, including batching mechanism and transmission time interval, directly impacts the implementation of backend

aggregation. Consequently, synchronization between embedded firmware and cloud processing logic is essential for maintaining consistent data management.

2.6 Research Gap Summary

From the previous sections, wireless sensor networks and IoT-based monitoring systems are already well developed in many aspects. Energy harvesting, low-power scheduling, local buffering, time synchronization, and cloud processing have all been investigated separately in previous research.

However, these components are often treated independently. In several energy-harvesting studies, the focus is mainly on hardware efficiency or power models, while transmission strategy and backend processing are not deeply discussed. On the other hand, cloud-oriented IoT platforms usually emphasize scalability and data analytics without considering the limitations of embedded nodes operating under unstable power conditions.

In real deployment scenarios, these aspects cannot be separated. For example, if a sensor node transmits data too frequently, energy consumption increases. If transmission is delayed, buffering mechanisms must handle larger data volumes. Similarly, timestamp errors caused by power interruption directly affect cloud-side aggregation accuracy. These interactions show that firmware behavior and backend logic influence each other.

Another issue is that reliability under intermittent connectivity is sometimes underestimated. While many systems assume stable communication, remote monitoring environments may experience unstable Wi-Fi or cellular connections. In such conditions, local storage and controlled retransmission are not optional optimizations but essential design elements.

Based on these observations, there is a need for a system design that considers energy constraints, buffering mechanisms, time recovery, and backend aggregation as interconnected parts rather than isolated features. This thesis focuses on integration, particularly from the perspective of firmware design and its interaction with cloud-side processing.

3. Embedded System Architecture

3.1 System Architecture Overview

The proposed system is designed as a battery-powered (solar charged) wireless monitoring architecture including three main parts: the embedded monitoring node, the communication layer and the cloud-based backend. These parts collaborate to gather environmental data, transfer it via a wireless network, and process it for storage and analysis.

At the device level, the system is built around an ESP32-S3 microcontroller and is powered by a rechargeable battery supported by a solar panel. In this process of the project, salinity, pH and temperature signals are provided through sensor emulation interfaces rather than physical sensing probes. The simulation allows testing of communication logic, buffering behavior, and time synchronization without requiring full sensor hardware integration. Although the sensing data are not collected by physical probes, the firmware architecture and transmission procedure are implemented as designed for actual deployment.

The device operates in a periodic measurement cycle. It remains in deep sleep most of the time to reduce energy consumption. At each interval, the microcontroller wakes up, generates or reads measurement values, attaches timestamp information, and stores the record in non-volatile storage. Data is not sent immediately after each measurement. Instead, multiple records are accumulated and transmitted together as a batch.

For communication, the system supports Wi-Fi and cellular connectivity depending on the deployment configuration. Batch-data is transmitted to a cloud endpoint using HTTP protocol. In case of network interruption, measurements remain stored in non-volatile storage until successful transmission.

On the server side, a serverless cloud function receives the uploaded batch-data. The backend validates the input, evaluates threshold conditions, and stores the records in a database. Besides raw records storage, statistical aggregation is implemented at different time intervals to support long-term analysis.

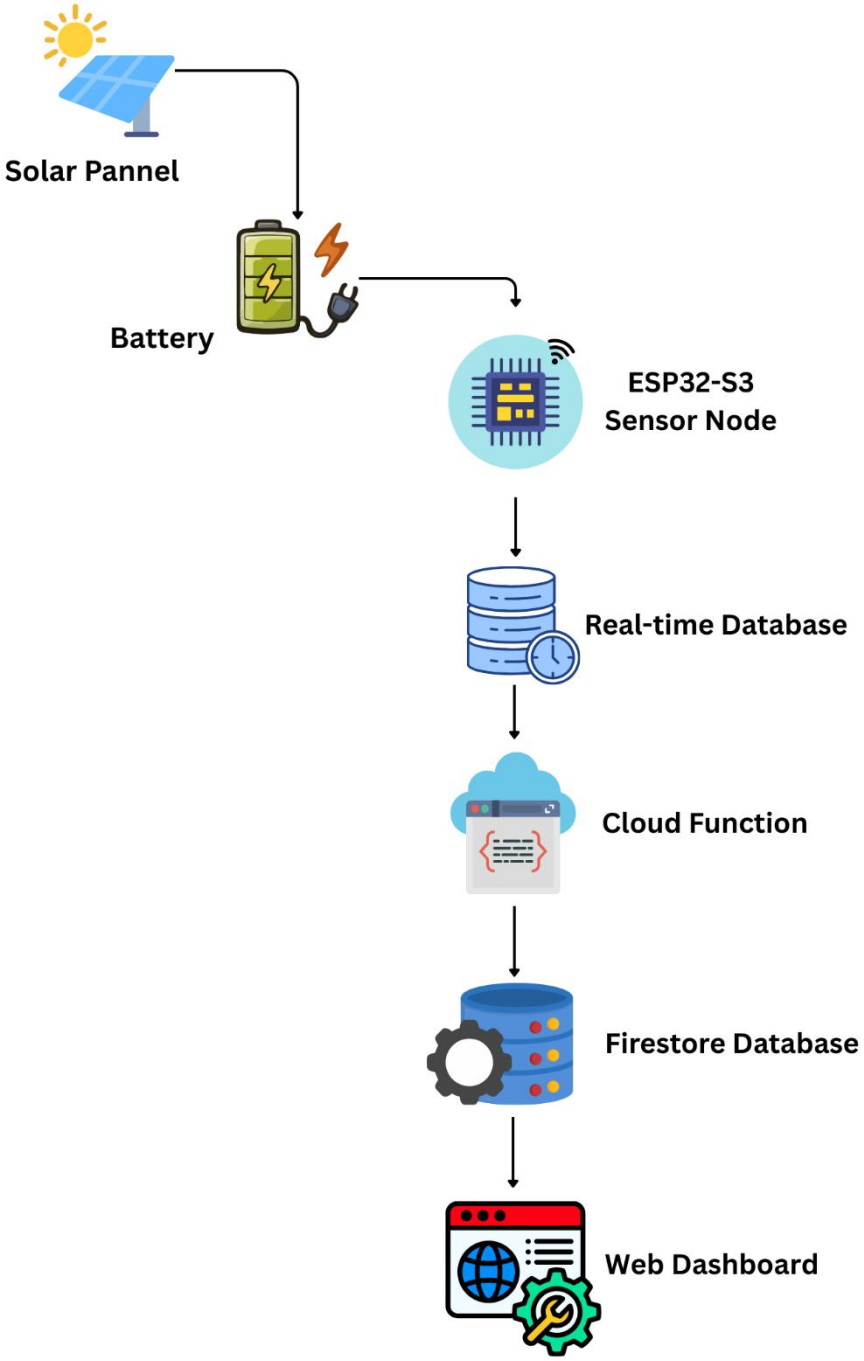

Figure 1.1 System Architecture Overview

3.2 Embedded Platform and Sensor Emulation Interfaces

The embedded node is constructed utilizing the ESP32-S3 microcontroller. This platform was chosen for its integration of processing capability, wireless connectivity, and low-power features in a single chip. The ESP32-S3 supports Wi-Fi connection and deep sleep mode, which are necessary for energy-efficient operation.

The power system consists of a rechargeable battery connected to a solar panel via a charge management module. The solar panel provides energy during daylight hours, while the battery maintains system operation when solar input is insufficient. Since solar energy availability is not constant, the firmware must optimize active time and reduce unnecessary communication events.

During this development phase, salinity and pH signals were provided through sensor emulation interfaces rather than fully integrated physical probes. In addition, temperature will be taken from real-time-clock module through I2C protocol with ESP32. The hardware design allows future integration of actual analog or digital sensors without major architectural changes. The emulation methods were used to validate system behavior under realistic operational conditions before full sensor deployment.

3.2.1 Analog ADC-Based Emulation

Analog signal emulation was implemented using potentiometer-based inputs. The ESP32-S3 reads voltage levels through its internal ADC channels. These voltage readings are scaled to represent salinity and pH measurements within predefined operating ranges.

This method allows verification of ADC sampling behavior, signal scaling logic, and data formatting procedures. By manually adjusting the potentiometer, different measurement values can be simulated to evaluate firmware response and storage behavior.

3.2.2 Modbus RTU Over RS485

Digital sensor emulation was executed with Modbus RTU communication over an RS485 interface. In this configuration, the ESP32 operates as a Modbus master and retrieves measurement values from holding registers. A third-party Modbus simulation application (pyModSlave) was utilized to provide adjustable register values. This setup allows validation of

communication timing, protocol parsing, and error handling without requiring a physical Modbus-based sensor module.

3.2.3 Real Time Clock Over I2C Protocol

The system also has an external RTC module (DS3231) that is connected by I2C protocol. RTC maintains accurate time across deep sleep cycles and power interruptions. Also, the module has a sensor that measures the temperature inside. The firmware reads this temperature once each cycle and saves it as an extra value for the ambient temperature. This reading shows the temperature inside the device enclosure, not the water temperature itself. However, it is assumed to be the water temperature sensor to replace the actual hardware sensor.

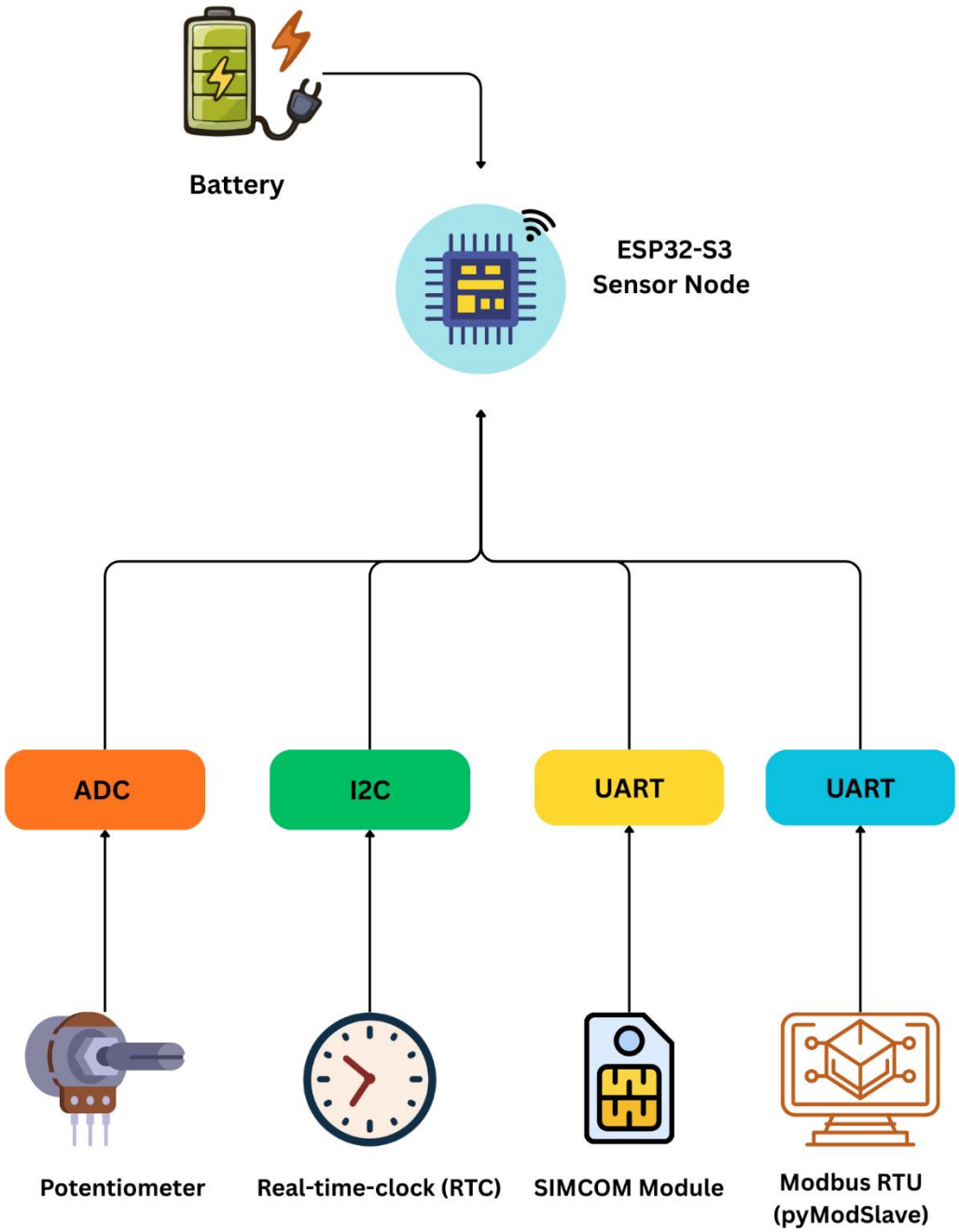

Figure 2.2 Embedded Platform and Sensor Emulation Interfaces

3.3 Firmware Architecture

The firmware is designed to support periodic environmental monitoring under energy-constrained conditions. Its structure is organized around a cyclic operating model combined with modular handling of data acquisition, local storage, and communication.

Instead of running continuously, the firmware follows a duty-cycled approach. The microcontroller remains in deep sleep for most of the time and only wakes up at predefined intervals. During each active period, the system performs a sequence of operations including measurement acquisition, timestamp handling, local buffering, and conditional data transmission.

At a high level, the firmware can be divided into three main functional layers: the operating cycle controller, the data acquisition layer, and the local buffering and transmission handler. These components interact sequentially during each wake-up cycle.

3.3.1 Operating Cycle

The operating cycle defines the overall behavior of the node. The sequence begins when the device wakes up from deep sleep through a time triggered interrupt.

After wake-up, the firmware initializes required peripherals and verifies system conditions such as time validity and storage availability. Measurement values are then acquired through the configured sensor interface. Once the data are obtained, the firmware attaches timestamp information and stores the record in non-volatile memory.

Data transmission is not performed after every measurement. Instead, the firmware evaluates whether the current sample count meets the configured upload condition. If the transmission threshold is satisfied and network connectivity is available, buffered data are sent in batch form. Otherwise, the system proceeds directly to the sleep stage.

Finally, the device re-enters deep sleep until the next scheduled wake-up interval. This cyclic model ensures predictable energy consumption and simplifies timing control.

3.3.2 Data Acquisition Layer

The data acquisition layer is responsible for retrieving measurement values from the selected sensor interface. During development, this includes analog ADC-based emulation, Modbus RTU communication, and temperature acquisition via an DS3231 RTC module connected to the ESP32.

For analog input, the firmware reads voltage levels from configured ADC channels and scales them into predefined salinity and pH ranges. For Modbus RTU input, the firmware operates as a master device and reads holding registers from a slave emulator over RS485. For temperature input, the firmware communicates with the DS3231 RTC through the ESP32 I2C interface.

The acquisition layer abstracts the source of the measurement data, allowing the rest of the firmware to process values in a consistent format regardless of the interface used.

3.3.3 Local Buffering Mechanism

To prevent data loss during network interruption, the firmware implements a store-and-forward buffering strategy. After each measurement cycle, records are written to non-volatile storage.

Instead of keeping only the latest reading, the firmware maintains a backlog of unsent records. When transmission conditions are met, multiple stored entries are grouped and uploaded together.

The buffering mechanism also includes basic overflow control to prevent uncontrolled growth of stored data under prolonged offline conditions. By combining periodic measurement scheduling with conditional batch transmission, the firmware balances energy efficiency and data reliability.

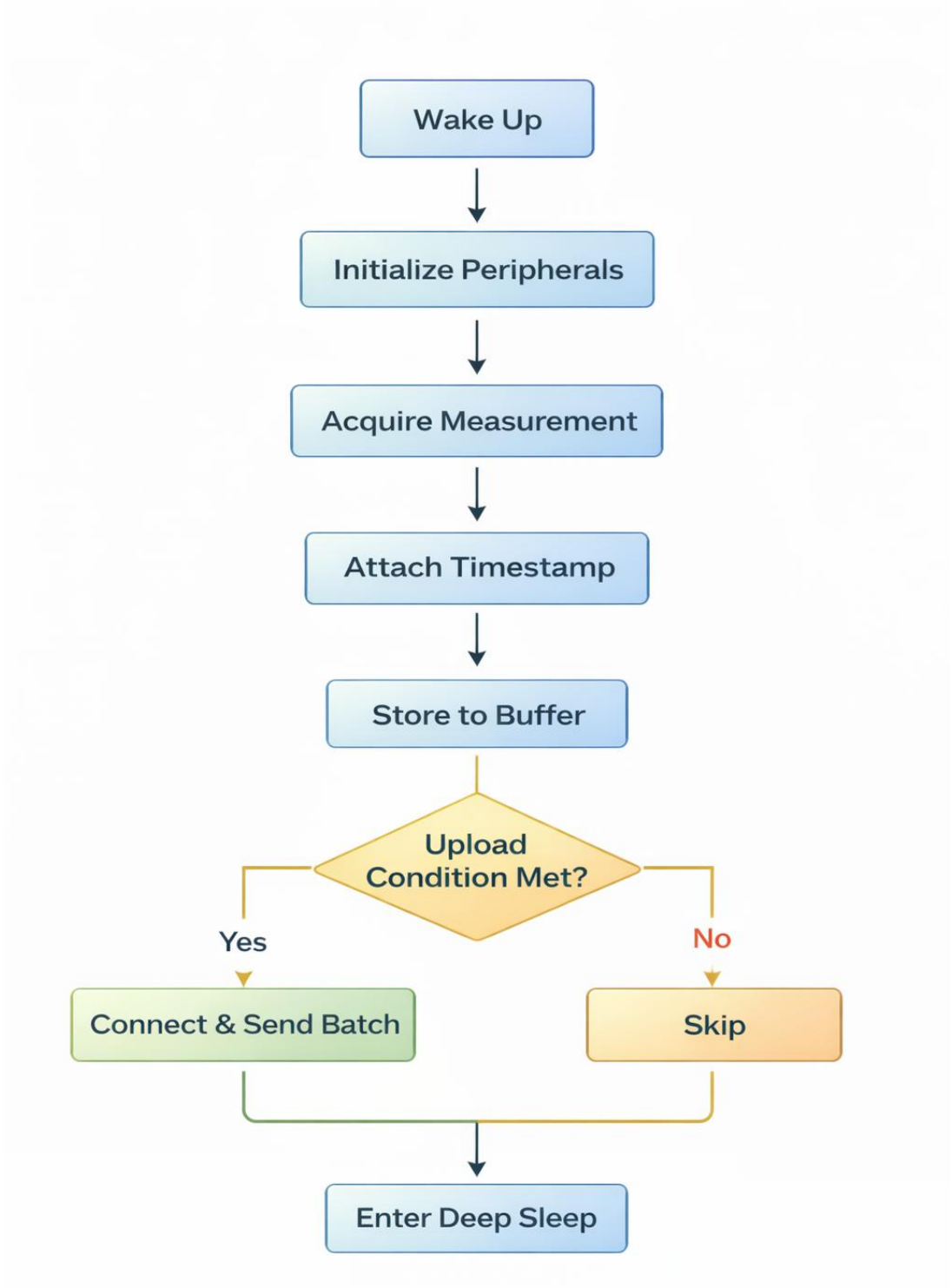

Figure 3.3 Firmware Operational Flow

3.4 Data Flow and Communication Strategy

The firmware is designed to separate data acquisition from data transmission. This separation allows the system to operate reliably even when network connectivity is unstable.

During each wake-up cycle, a measurement record is generated and written to non-volatile storage. The record typically includes scaled salinity and pH values, optional raw values, battery status, and timestamp information. Once stored locally, the data is considered secured regardless of network availability.

Transmission is handled conditionally. Instead of uploading every measurement immediately, the firmware evaluates whether a predefined transmission interval or sample threshold has been reached. Only when this condition is satisfied does the device attempt to establish network connectivity and send buffered data.

This strategy reduces the number of connection events, which are typically the most energy-consuming operations in wireless systems. By grouping multiple records into a single batch, communication overhead is minimized and overall system efficiency is improved.

The firmware supports different connectivity modes depending on deployment configuration. When Wi-Fi is available, the device connects to a configured access point and performs HTTP-based data upload. In other configurations, cellular connectivity may be used. In both cases, the transmission process follows the same logical structure: establish connection, transmit batch data, confirm response, and update local buffer status.

If transmission fails due to network interruption or server-side error, the firmware does not discard the data. Instead, the records remain stored in local memory and will be retried during the next transmission opportunity. This store-and-forward approach ensures that temporary connectivity issues do not result in permanent data loss.

By decoupling measurement scheduling from communication events, the firmware maintains predictable energy behavior while preserving data reliability under intermittent network conditions.

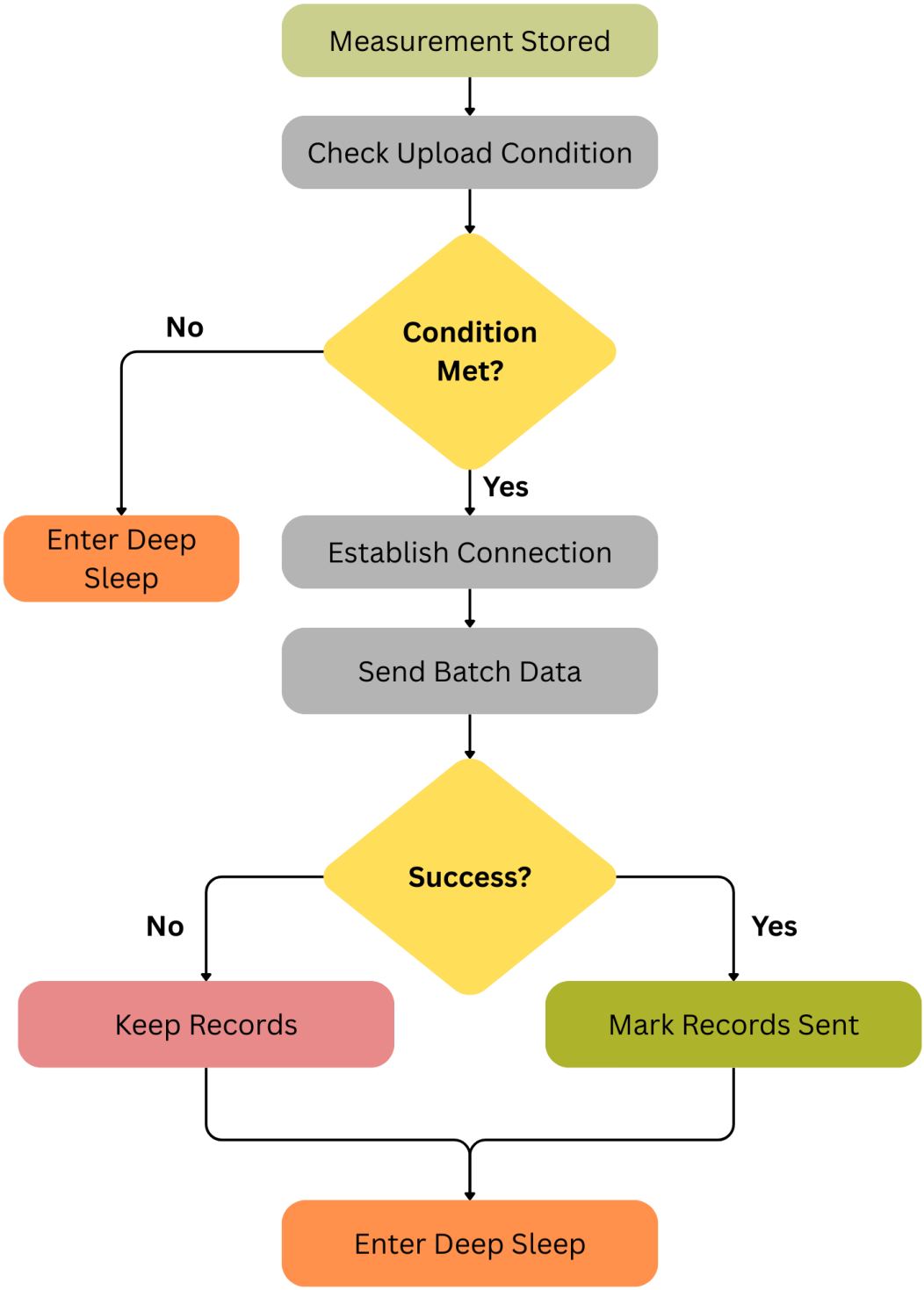

Figure 4.4 Data Transmission Strategy

3.5 Power Management Strategy

Since the system operates on a rechargeable battery supported by solar energy, power management plays a central role in firmware design. Unlike mains-powered devices, the node must operate under fluctuating energy availability. Solar input depends on environmental conditions, and battery capacity is limited. As a result, the firmware is structured to minimize active processing time and reduce unnecessary communication events.

The primary energy-saving mechanism is the use of deep sleep scheduling. The ESP32-S3 remains in deep sleep for most of its operating cycle and only wakes up at predefined intervals to perform measurement and data handling tasks. This duty-cycled operation significantly reduces average current consumption compared to continuous operation.

The duration of the active period is kept as short as possible. During each wake-up event, the firmware performs measurement acquisition, timestamp processing, local buffering, and conditional transmission. If the upload condition is not satisfied, the system skips network initialization and returns to sleep immediately. Avoiding frequent network reconnection is especially important because wireless communication typically consumes substantially more energy than sensor reading or data processing.

Batch-based transmission further contributes to energy efficiency. By accumulating multiple records before attempting communication, the number of connection cycles is reduced. This design choice balances data freshness with power consumption, which is particularly relevant in solar-powered deployments.

During development, battery behavior was also emulated within the firmware to evaluate system response under varying energy conditions. Although this emulation does not represent a full electrochemical battery model, it provides a practical method for validating duty-cycle logic and conditional operation before final hardware integration.

Overall, the power management strategy is based on three principles: minimizing active time, limiting communication frequency, and ensuring reliable operation under intermittent energy availability. These principles guide the firmware structure described in the previous sections.

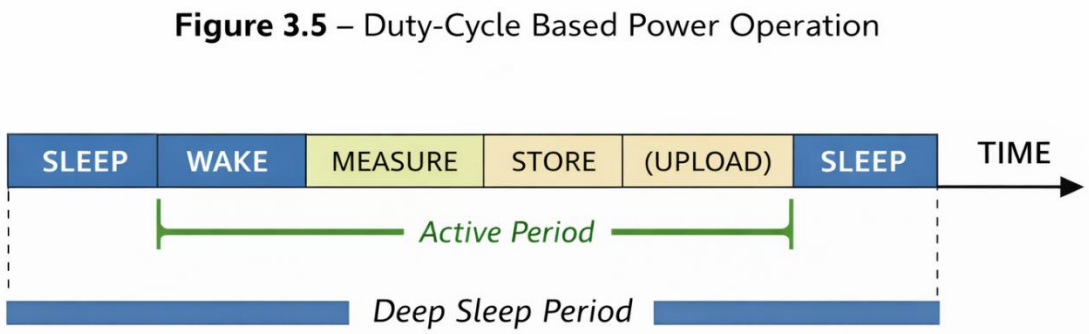

Figure 5.5 Duty-Cycle Based Power Operation

4. Firmware Design and Implementation

4.1 Firmware Design Requirements

The firmware was designed under several practical constraints derived from the characteristics of battery-powered wireless monitoring systems. The firmware was developed using the Arduino framework for ESP32, mainly because it provides fast iteration and many ready-to-use libraries. These constraints influenced both the structural organization and the operational behavior of the embedded software.

Firstly, energy availability is limited and variable. Since the node operates on a rechargeable battery supported by solar energy harvesting, continuous active operation is not feasible. The firmware must therefore minimize active runtime and reduce high-power activities such as wireless communication.

Secondly, network connectivity cannot be assumed to be stable. In remote deployments, Wi-Fi or cellular signals may become temporarily unavailable. The firmware must tolerate such interruptions without losing measurement data. This requirement leads to the adoption of a local store-and-forward mechanism.

Thirdly, timestamp integrity must be preserved despite possible power resets. In battery-powered systems, deep discharge or unexpected resets may invalidate real-time clock values. Therefore, the firmware must include time synchronization and recovery mechanisms.

Fourthly, overhead communication must be controlled. Establishing wireless connections consumes significantly more energy than measurement or local processing. The firmware must therefore support batch transmission instead of sending every measurement immediately.

Finally, the firmware must remain modular and hardware independent. Since salinity, pH, temperature inputs were emulated during development through ADC, Modbus RTU and I2C interfaces, the acquisition layer must abstract hardware details from the buffering and communication layers. This design ensures future sensor integration without major architectural changes.

Based on these constraints, the firmware implementation follows a duty-cycled execution model combined with persistent local storage and conditional communication logic.

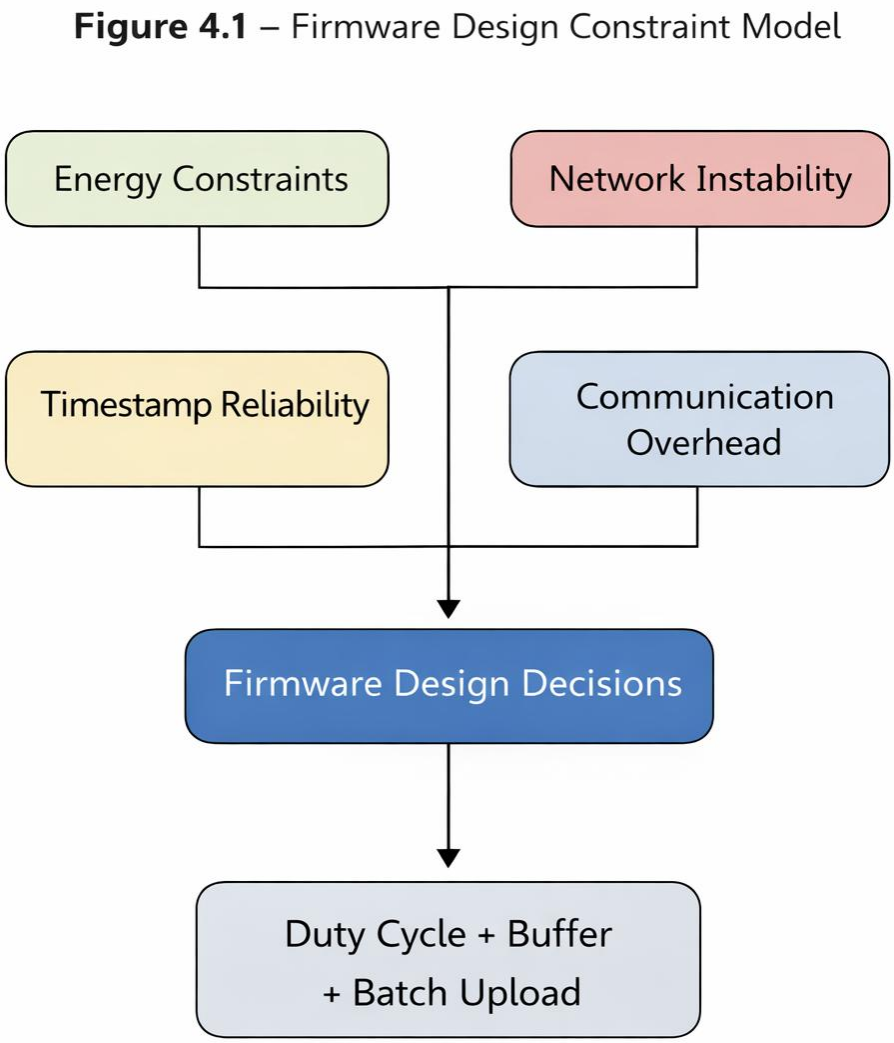

Figure 6.1 Firmware Design Overview

4.2 Implementation of Operating Cycle

The firmware operating cycle is implemented using a timer-based deep sleep mechanism provided by the ESP32 platform. Instead of executing continuous background tasks, the system performs a complete measurement cycle during each wake-up event and then returns to deep sleep.

After the configured sleep interval expires, the microcontroller wakes up by external RTC and begins execution from the start of the program. During this active period, the firmware initializes required peripherals and prepares the system for measurement acquisition.

The operating sequence consists of the following steps:

1. Initialize communication and storage interfaces.
2. Acquire measurement values from the configured sensor interface.
3. Format and prepare the measurement record.
4. Store the record in non-volatile storage.
5. Evaluate whether the transmission condition has been met.

Transmission is controlled by a configurable upload interval. The firmware maintains an internal counter to determine when a batch upload should be attempted. If the condition is satisfied, the system establishes network connectivity and transmits accumulated records. Otherwise, the communication step is skipped to conserve energy.

After completing the active tasks, the firmware configures the next timer-based wake-up event and enters deep sleep mode. All wireless interfaces are disabled before entering sleep to minimize current consumption.

This execution model ensures predictable timing behavior while maintaining energy efficiency through duty-cycled operation.

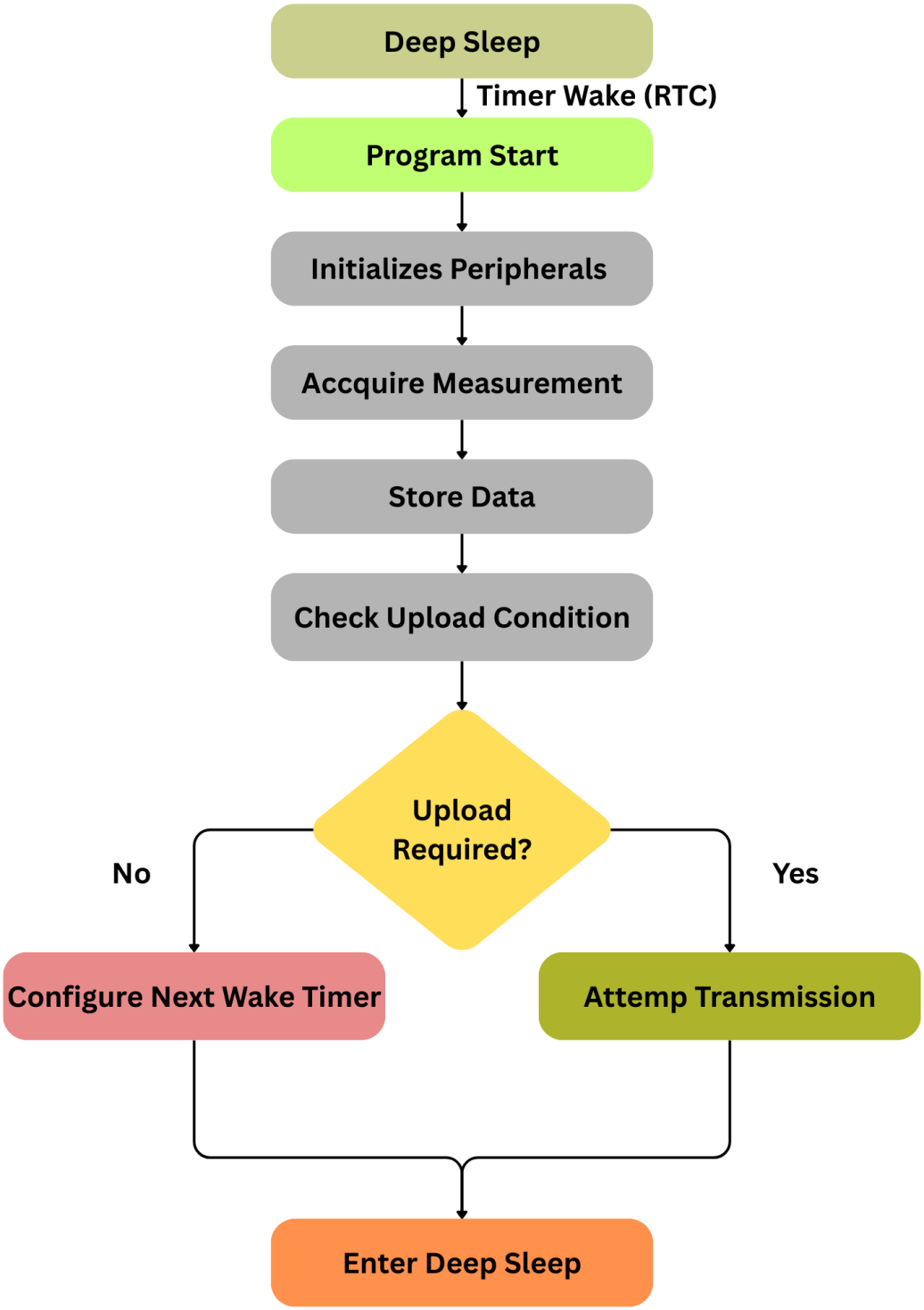

Figure 7.2 Wake-up Execution Cycle

4.3 Implementation of Data Acquisition Logic

The data acquisition stage is responsible for producing a consistent set of measurement values for the rest of the firmware pipeline. Regardless of how the values are obtained, the output of this stage is normalized into the same record format before being stored and later transmitted.

During development and integration, two acquisition approaches were used: analog voltage sampling through the ESP32 ADC channel and Modbus RTU register reading through an RS485 interface, and temperature reading via the DS3231 RTC module connected to the ESP32 over I2C. The final deployment-oriented workflow primarily relies on Modbus RTU for measurement retrieval while ADC-based inputs were used to validate the firmware behavior when sensor hardware was not yet available. In parallel, the RTC provides an ambient temperature, which is considered a water temperature substitute for the real sensor.

4.3.1 Analog ADC-Based Input (Development Support)

For the analog interface, the firmware samples a voltage level using the ESP32 ADC by using potentiometers.

Potentiometer produces a voltage between 0 V and approximately 3.3 V. Then the ESP32-S3 reads this voltage using analogReadMilliVolts(), which returns a millivolt value. The sampled value is averaged across multiple readings to reduce noise and improve stability. The average voltage is then mapped into an application-specific range representing salinity or pH:

- Salinity: 0–3245 mV → 0–100 ppt (depend on demo range)
- pH: 0–3245 mV → 0–14

This approach provides a controllable input signal that is useful for validating firmware logic such as buffering, batching, and timestamping. Since the analog source is an emulation input, the mapping is treated as a practical scaling method rather than a calibration model for a real probe.

4.3.2 Modbus RTU Measurement Retrieval (Primary Interface)

For the Modbus RTU interface, the firmware acts as a Modbus master and retrieves measurement values from a slave device over RS485. The acquisition sequence includes:

- Initializing the UART port with Modbus parameters
- Issuing a request to read holding registers
- Validating the response status and registering content
- Converting register values into engineering units.

In the current system stage, salinity and pH are represented as register values supplied by an external source (either a sensor module or a register emulator). The firmware treats these values as the primary measurement inputs and focuses on reliable transport, error handling, and consistent formatting.

To handle communication errors, the firmware detects failed Modbus transactions and assigns safe fallback values for the current cycle. This ensures that the operating cycle and buffering mechanism remain deterministic even when the sensor interface is temporarily unavailable.

4.3.3 I2C Interface Input

The RTC DS3231 connects to the ESP32-S3 using I2C interface (commonly at address 0x68). The firmware reads the DS3231's internal temperature during each acquisition cycle. These registers give a signed temperature reading with a resolution of 0.25 °C. The raw register data is changed to degrees Celsius and then used as a standard measurement with salinity and pH.

4.3.4 Output Normalization and Record Preparation

After acquisition, values are normalized to a consistent numeric format before being passed to the storage layer. This step may include rounding to a fixed number of decimal places, applying scaling factors, and attaching metadata such as battery status and timestamps. Normalization keeps the later stages (buffering and cloud ingestion) independent from acquisition-specific details.

4.3.5 Development Iteration: Initial Modbus TCP Testing (Prototype Development)

During the early development phase, a Modbus TCP interface was briefly evaluated to validate register-based data acquisition using a network-based emulator. However, due to

deployment constraints and the preference for RS485-based communication in embedded field environments, the system architecture was later standardized on Modbus RTU.

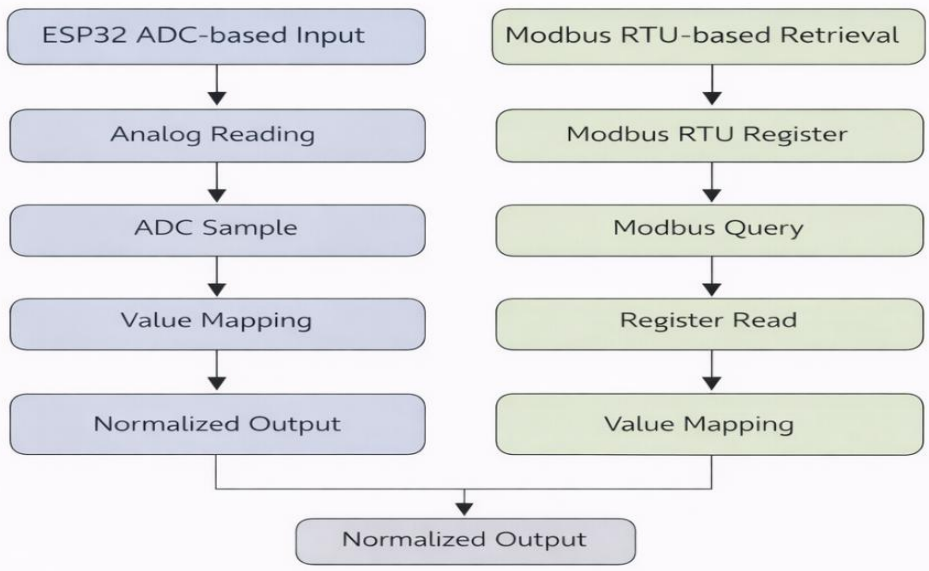

Figure 4.3 – Data Acquisition Pathways

Figure 8.3 Data Acquisition Pathways (ADC vs Modbus RTU)

4.4 Local Data Buffering Implementation

Reliable data delivery is a critical requirement for solar-powered wireless sensor nodes operating under intermittent connectivity. To prevent data loss during temporary network outages, the firmware implements a local buffering mechanism using non-volatile storage.

Instead of transmitting each measurement immediately, records are first stored locally. Transmission is attempted only when predefined upload conditions are satisfied. This store-and-forward strategy ensures that measurements are preserved even if wireless communication fails during a given operating cycle.

4.4.1 Preferences-Based Queue Mechanism (Primary Implementation)

In the final firmware implementation, the buffering mechanism is built on the ESP32 non-volatile storage (NVS) using the Preferences library.

Each measurement record is serialized into a JSON-formatted string and stored with an incremental key index. A persistent counter tracks the number of queued records. During a transmission event, all stored records are read sequentially and packaged into a batch payload. If the upload is successful, the stored entries are cleared from memory.

This approach provides several advantages:

- Data persistence across deep sleep cycles
- Recovery after temporary network failure
- Controlled memory usage through indexed storage
- Simplicity of implementation within the ESP32 NVS framework

Because NVS survives power cycling, the queue remains intact even if the device resets unexpectedly.

4.4.2 Batch Transmission and Queue Clearing Logic

The firmware does not delete records individually after each upload attempt. Instead, the queue is cleared only when the server confirms successful reception.

The transmission procedure follows this logic:

1. Retrieve all stored records.
2. Build a batch JSON payload.
3. Attempt HTTP transmission.
4. If HTTP response is successful → clear queue.
5. If transmission fails → retain queue for next cycle.

This strategy prevents partial data loss and avoids inconsistencies between local storage and cloud records.

4.4.3 LittleFS-Based Buffering (Prototype Development)

During early development, a file-based buffering mechanism using LittleFS was evaluated. This approach stored each record as a separate line within a local file and maintained metadata (large backlogs) to track upload progress.

Although the file-based system provided fine-grained control over backlog management and compaction, the final deployment-oriented implementation adopted the NVS-based queue for simplicity and reduced filesystem overhead.

The LittleFS exploration contributed to validating buffer integrity and recovery mechanisms but is not part of the final operational firmware.

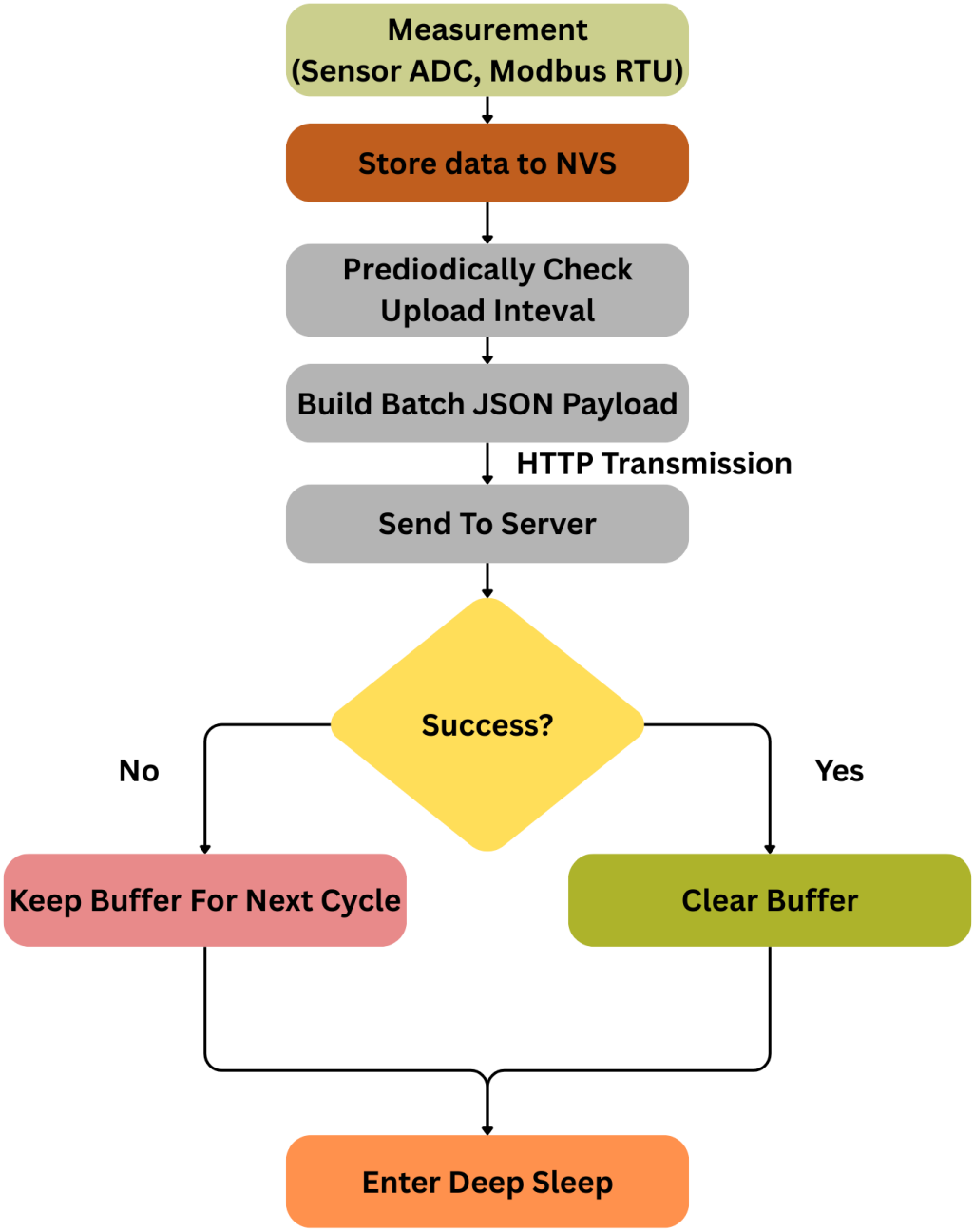

Figure 9.4 Store-and-Forward Buffering Workflow

4.5 Communication and Cloud Transmission Implementation

Wireless communication represents the most energy-intensive stage of the firmware operating cycle. Therefore, transmission is carefully controlled and executed only when necessary. The communication module is responsible for establishing network connectivity, delivering batch payloads to the cloud endpoint, and handling transmission outcomes.

The firmware includes two different upload implementations: Wi-Fi HTTPS upload and cellular HTTPS upload via SIMCOM A7682S module. Both versions share the same payload structure and batch-upload concept, which simplifies the backend.

4.5.1 Network Initialization and Connection Handling

Depending on deployment configuration, the firmware supports either Wi-Fi or cellular connectivity. In both cases, network interfaces remain disabled during deep sleep and are activated only when transmission is required.

Before attempting data upload, the firmware:

1. Enables the wireless interface.
2. Initiates connection to the configured access point or cellular network.
3. Waits for connected confirmation within a defined timeout.

If the connection cannot be established within the timeout period, the transmission attempt is aborted, and the system returns to deep sleep. No buffered data is deleted in this case.

This time-out-based strategy prevents excessive energy consumption caused by prolonged connection attempts under poor signal conditions.

4.5.2 Batch Payload Construction

Once network connectivity is established, the firmware retrieves all buffered records and constructs a single JSON payload.

The payload includes:

- Device identifier
- An array of measurement records
- Timestamp
- Battery status information.

Batch transmission reduces protocol overhead compared to sending each measurement individually. By grouping multiple records into a single HTTP request, the firmware minimizes the total active communication time.

4.5.3 Wi-Fi Upload via HTTPS

For Wi-Fi uploading, the firmware uses an HTTP POST request to send the batch payload to a predefined cloud endpoint. After sending the request, the system waits for the server response.

The transmission result is evaluated as follows:

- If the HTTP response indicates success → buffered records are cleared.
- If the response indicates failure or timeout → buffered records are retained.

After a successful response, the queue is cleared. The firmware then disconnects Wi-Fi and turns off the Wi-Fi interface.

This confirmation-based mechanism guarantees that no data is removed unless acknowledged by the backend.

4.5.4 Cellular Upload via SIMCOM A7682S

The cellular firmware uses a SIMCom A7682S LTE Cat 1 modem controlled via UART. TinyGSM handles network attachment and APN setup, while HTTPS uploads are executed using SIMCom's HTTP/SSL AT-command workflow (e.g., HTTPINIT, HTTPPARA, HTTPDATA, HTTPACTION).

Because the modem's built-in HTTP stack is limited, the firmware sends an HTTP POST with the query parameter x-http-method-override=PATCH to achieve Firebase merge behavior similar to a Wi-Fi PATCH upload.

4.5.5 TCP Modbus Implementation (Prototype Development)

While waiting for the printed circuit board to be delivered, an earlier firmware version was developed that used Modbus TCP instead of Modbus RTU. In that version, the ESP32-S3 connected to pyModSlave over Wi-Fi (Modbus TCP) and read three simulated registers (salinity,

temperature, pH). The firmware also implemented a file-based buffer using LittleFS and sent JSON records (built with ArduinoJson) to a custom HTTP Cloud Function endpoint (ingestBatch).

This approach worked, but it became less suitable for the final project for two reasons:

- It added complexity (TCP register reads + file management + custom ingestion endpoint).
- It was not aligned with the final hardware direction (RS-485 is more common for industrial sensors).

Therefore, this version is considered deprecated and is not used in the final thesis. However, it is included in Appendix C.3 because it shows alternative design ideas and can be a starting point for future improvements.

4.5.6 Energy Considerations During Communication

Wireless communication consumes significantly more current than sensing or local storage operations. Compared to Wi-Fi, the cellular connection setup is slower and more sensitive to network conditions. Therefore, the firmware ensures that:

- The network interface is enabled only during upload.
- Transmission is performed in batches rather than per measurement.
- Connection retries are limited by timeout constraints.
- The device returns to deep sleep immediately after transmission.

These measures contribute to extending battery lifetime in battery-powered operation, particularly during periods of low energy availability.

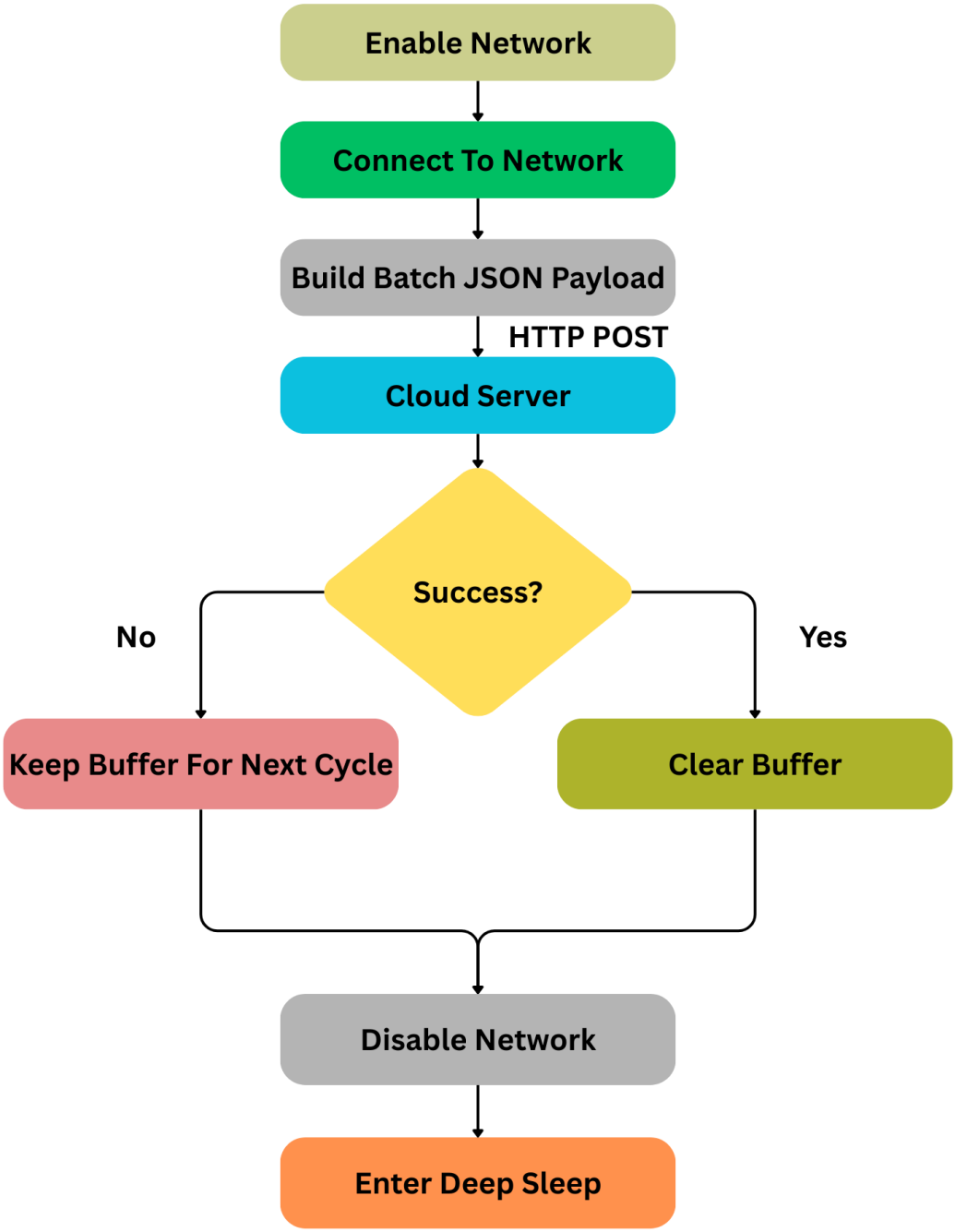

Figure 10.5 Communication and Transmission Sequence

4.6 Time Synchronization and Timestamp Handling

Accurate timestamping is essential for environmental monitoring systems, particularly when long-term data aggregation and trend analysis are performed on the cloud platform. In the proposed firmware, timestamp management is implemented to ensure temporal consistency across deep sleep cycles and intermittent network connectivity.

4.6.1 Initial Time Synchronization

When network connectivity becomes available, the firmware synchronizes system time using an NTP (Network Time Protocol) server. The synchronized epoch time is stored internally and used as the reference for subsequent measurements.

Time synchronization is not performed at every operating cycle. Instead, it is executed only when necessary to reduce communication overhead. Once synchronized, the firmware maintains a running epoch value during subsequent cycles.

4.6.2 Timestamp Progression Across Deep Sleep

Because the ESP32 restarts program execution after exiting deep sleep, the firmware maintains time continuity by incrementing the stored epoch value according to the configured measurement interval.

After each measurement cycle:

- The current timestamp is attached to the record.
- The epoch value is increased by the measurement interval.
- The updated value is retained for the next cycle.

This approach avoids the need to re-synchronize with the NTP server at every wake-up event, reducing network usage and energy consumption.

4.6.3 Handling Communication or Synchronization Failure

If network connectivity is unavailable during startup, NTP synchronization cannot be performed immediately. In this case, the firmware temporarily assigns a default or previously stored epoch value and continues data acquisition.

Once network connectivity is restored, time synchronization is performed again to realign the internal clock. This mechanism ensures that data collection is not blocked by temporary synchronization failure.

4.6.4 Timestamp Integration in Batch Payload

Each measurement record includes:

- Measured timestamp (epoch format)
- Device identifier
- Battery status
- Sensor values

Using epoch-based timestamps ensures compatibility with backend aggregation functions, such as 5-minute, hourly, and daily statistical grouping.

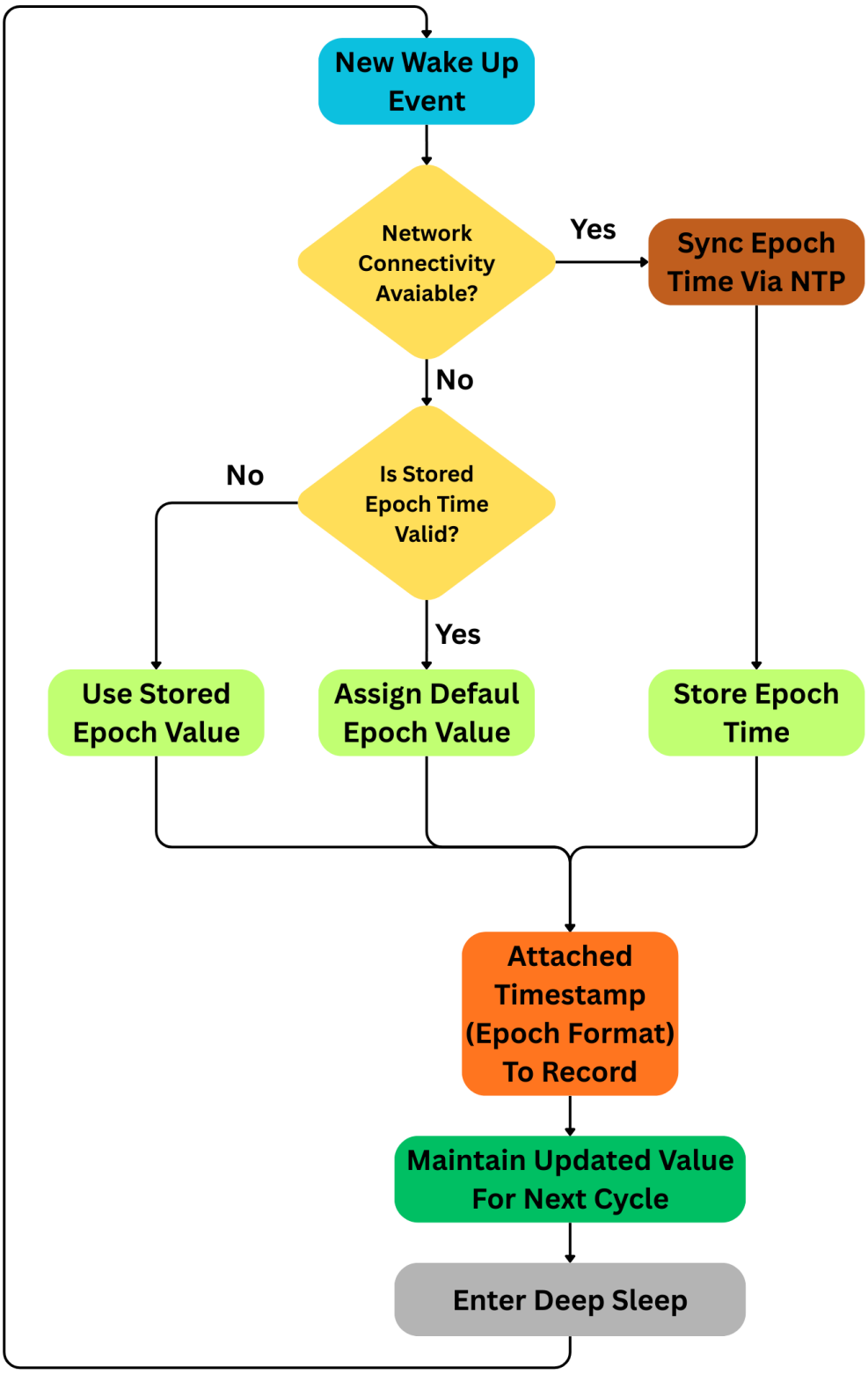

Figure 11.6 Timestamp Handling Across Deep Sleep Cycles

4.7 Firmware Reliability and Error Handling

A wireless sensor node that runs on solar power must be able to handle variable network conditions, power interruptions, and communication problems. Hence, the firmware has been designed with fallback methods to ensure stable long-term operation.

4.7.1 Network Connection Failure Handling

Before each transmission attempt, the firmware sets up the wireless interface and attempts to connect to the network within a defined timeout period.

If the connection fails:

- No buffered data is deleted.
- No longer retry loops are executed during the same cycle.
- The system turns into deep sleep immediately.

This strategy prevents excessive energy consumption caused by repeated connection attempts under poor signal conditions.

4.7.2 Transmission Failure Handling

The firmware checks the server's response after submitting an HTTP POST request.

- If the response says it worked → the queue is cleared
- If the response says it failed or timed out → the queue stays the same.

This confirmation-based deletion system makes sure that data is never deleted without the backends' clear permission.

4.7.3 Sensor Interface Failure Handling

During development, the system used simulated data sources from Modbus RTU connection and ADC-based signal injection. If a reading attempt fails:

- The firmware does not crash.
- If any invalid data is stored in storage → the cloud backend will filter it later
- The system continues with the next duty cycle

4.7.4 Safe Deep Sleep Recovery

After each cycle, the system transitions into deep sleep mode. Upon wake-up, the firmware resets the necessary peripherals and gets the runtime state variables back from non-volatile storage.

This design ensures:

- Execution is certain following each wake-up
- Time moves forward in a consistent way.
- Preservation of unsent measurements.

The firmware avoids undefined situations that often happen with long-running embedded loops since execution always starts from a known entry point.

4.8 Firmware Summary

This chapter described the firmware implementation for a battery-powered ESP32-S3 sensor node. The most important design choice is the duty-cycled workflow using deep sleep and periodic batch upload. Two communication options were implemented: Wi-Fi and cellular via SIMCOM A7682S connectivity. The firmware also supports sensor simulation using potentiometers, Modbus and RTC, which allow development without physical salinity, pH and temperature hardware sensors.

In the next chapter, the cloud backend function is described, which processes the uploaded data and prepares it for dashboard visualization.

5. Cloud Backend Processing Design

5.1 Cloud System Architecture

On the cloud side, this project uses Firebase as the main platform because it provides database services and serverless backend functions in one ecosystem. The services used in our project are:

1. Firebase Realtime Database (RTDB): used as a simple ingestion endpoint for the ESP32 firmware
2. Cloud Functions (backend): used to process new records and migrate data.
3. Cloud Firestore Database: used as the structured database for the web dashboard

In the current implementation, the cloud backend is organized into two sequential stages. Measurement data generated by the embedded node is first pushed to Firebase Realtime Database. A Cloud Function is configured to monitor the corresponding database path; whenever new data is written, the function retrieves the record, performs validation and basic status checks, and then stores valid entries in Firestore Database.

With this approach, the embedded device only handles simple data transmission. ESP32 communicates with RTDB through lightweight REST-based HTTP requests, which keeps the firmware implementation straightforward. More computational tasks such as data validation, condition evaluation, and aggregation are handled within the cloud environment. This separation reduces processing complexity on the device side while allowing backend logic to be modified or extended without requiring firmware updates.

The backend therefore consists of the following components:

1. Realtime Database (data ingestion layer)
2. Event-triggered Cloud Function (bridge processor)
3. Firestore database (persistent structured storage)
4. Aggregation collections (multi-interval statistics)

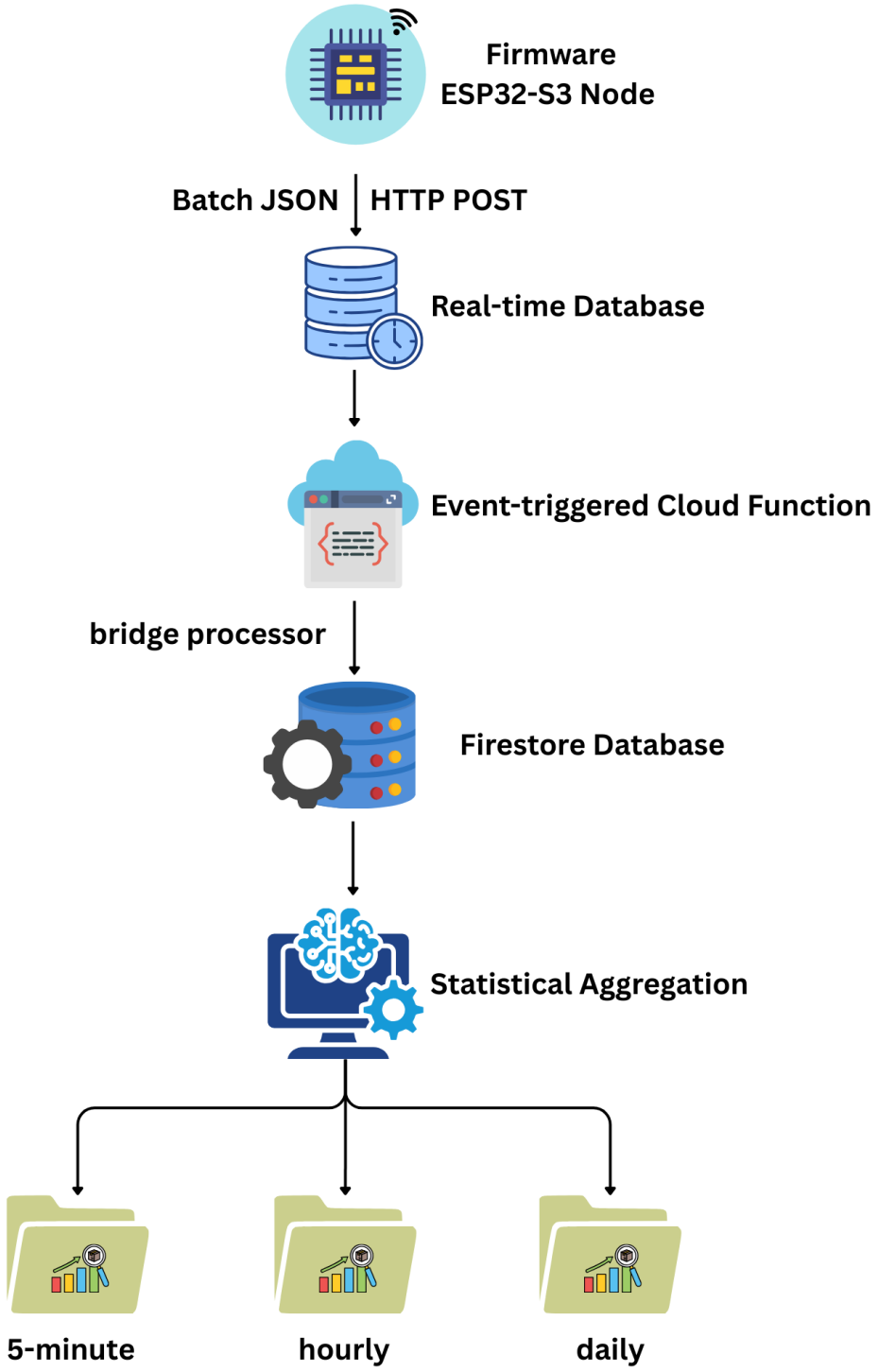

Figure 12.1 Cloud Backend Architecture

5.2 Realtime Database Processing Pipeline

5.2.1 Data Upload from ESP32 to Realtime Database

The embedded node delivers measurement data to a known RTDB endpoint via an HTTP PATCH request. Each upload contains:

- Device identifier
- Sensor values (salinity, temperature, pH)
- Battery percentage and voltage
- Timestamp (epoch format)

RTDB was chosen as the ingestion layer because it provides easy REST-based writing without requiring complex authentication function. This reduces firmware complexity and communication overhead for the ESP32. That means, the device does not directly communicate with Firestore database.

5.2.2 Event-Triggered Bridge Function

A cloud function is configured to trigger whenever a new entry is created under the RTDB path: "/data/{pushId}". This trigger is supported by Cloud Functions for Firebase v2 database events.

When activated, the function follows these steps:

1. Reads the newly inserted payload.
2. Extracts and validates required fields.
3. Normalizes numeric values.
4. Evaluates threshold conditions.
5. Writes processed data into Firestore database.

If validation fails:

- The record is logged.
- No Firestore commit occurs

A useful design detail is idempotency. In Firestore, the document ID is set to the same {pushId} value from RTDB. For instance, if the same RTDB record is processed again, the Firestore will overwrite the same document instead of creating duplicates.

This trigger function method keeps the firmware lightweight while ensuring that backend processing is centralized and consistent.

5.2.3 Firestore Structured Storage

After validation, the bridge function writes data into Firestore database under a device-specific collection: “devices/{deviceId}/readings/{autoDocId}”

Each document contains:

- A “readings” collection (raw measurements)
- A “stats/latest” document
- Aggregation collections (5-minute, hourly, daily)
- Computed status (OK / WARN / ALERT)
- Timestamp

Splitting RTDB from Firestore database enhances architectural clarity:

- RTDB serves as raw ingestion buffer.
- Firestore serves as structured analytics database.

5.2.4 Multi-Interval Statistical Aggregation

Besides storing raw readings, the backend maintains aggregated statistics to support fast dashboard queries. Instead of querying and computing averages on the client side every time, the server stores pre-computed values for fixed time windows.

Aggregation logic updates statistical summaries at several temporal resolutions while executing the same cloud operation:

- 5-minute buckets

- Hourly buckets
- Daily buckets

For each bucket contains:

- Count
- Sum
- Minimum
- Maximum
- Computed average

Aggregation occurs on the server side to ensure consistency and avoid unnecessary computation at the embedded side.

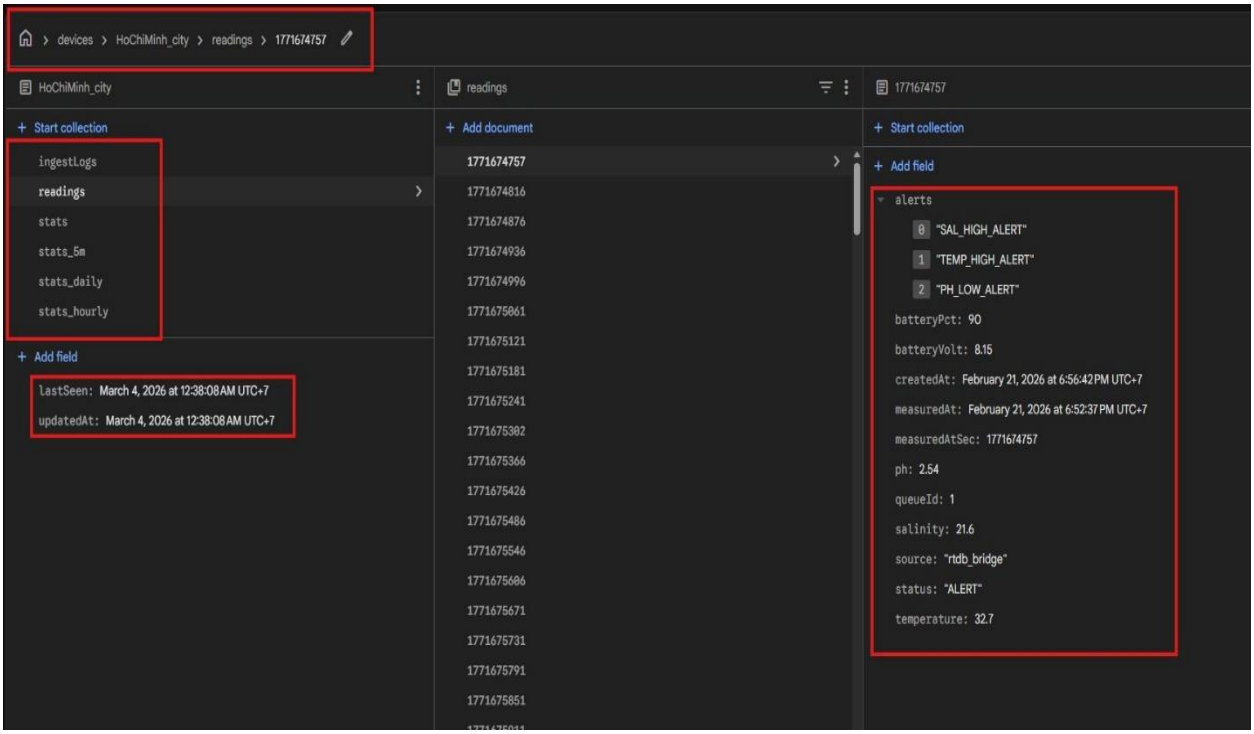

Figure 13.2 Statistical Aggregation

5.3 Threshold Evaluation Logic

The cloud function does a threshold evaluation to sort measurements into operational states.

Each record is evaluated against predefined ranges for:

- Salinity
- Temperature
- pH
- Battery percentage

Each parameter has defined acceptable bounds:

- $S_{min} \leq S \leq S_{max}$
- $P_{min} \leq P \leq P_{max}$
- $B_{min} \leq B \leq B_{max}$
- $T_{min} \leq T \leq T_{max}$

Status levels:

- **OK**: Within acceptable bounds
- **WARN**: Near threshold boundary (outside OK but inside WARN becomes WARN; outside WARN becomes ALERT)
- **ALERT**: Outside WARN range

Parameter	OK range	WARN range
Salinity (ppt)	1.5 – 3.0	1.0 – 3.5
Temperature (°C)	20 – 30	18 – 32
pH	6.0 – 7.5	5.5 – 8.0
Battery (%)	≥ 40	≥ 25

Table 1.3 Thresholds range

To assist in monitoring dashboards and decision systems, alert codes are saved with the measurement. By doing evaluation in the backend instead of in firmware, the system can manage rules from one place and change thresholds without changing firmware.

5.4 Database Organization

The Firestore database structure is organized per device (node):

```
devices/{deviceId}
    ├ readings/
    ├ stats/latest
    ├ stats_5m/
    ├ stats_hourly/
    ├ stats_daily/
```

This structure supports:

- Efficient querying of latest data
- Long-term trend visualization
- Reduced cost of aggregation queries

Entries records and aggregated statistics are intentionally separated to optimize read operations.

5.5 Development Utilities (Migration Function)

During system transition from RTDB storage to Firestore database storage, a migration utility was implemented.

Its purpose:

- Read historical RTDB entries
- Reformat schema
- Insert old data into Firestore
- Preserve timestamps

This function is not part of operational pipeline and was used only during backend restructuring.

5.6 Direct HTTP Ingestion Pipeline (Prototype Development)

During the early stages of development, a different architecture was used that allowed the ESP32 to send batched measurements directly to a cloud HTTP endpoint “ingestBatch”. This method made the system test for batch transmission, authentication keys, and aggregation logic without relying on RTDB triggers.

This approach had some advantages:

- It allows more control over validation and authentication.
- The backend can reject bad data before it reaches RTDB.

For the final deployment of architecture presented in this thesis, the RTDB pipeline was chosen because:

- It simplifies firmware networking logic.
- It allows easier debugging of raw incoming data.
- It separates ingestion from processing.
- It provides an intermediate data buffer.

5.7 Cloud Backend Summary

The backend in this thesis takes raw uploads from devices and turns them into structured Firestore documents. The Cloud Function design includes validation, threshold classification, latest-reading updates, and multi-scale aggregation. This architecture keeps the ESP32 firmware lightweight and moves most data processing to the server side, which is easier to update and maintain.

In the next chapter, the experimental evaluation results are presented.

6. Experimental Evaluation

6.1 Experimental Setup

This chapter evaluates how the proposed system performs under practical operating conditions. The goal is to verify that the firmware and cloud backend work correctly together, rather than to measure laboratory-level sensor accuracy.

An ESP32-S3 development board running on a rechargeable battery system was used for the thesis. Deep sleep was enabled in between cycles, and the firmware was set up with a scheduled measurement interval. Because physical salinity and pH sensors were not available during development, measurement values were collected using signal emulation. A potentiometer-based analog input and Modbus RTU connection used to simulate sensor values. This configuration allowed the firmware logic, buffering mechanism, and communication pipeline to be evaluated in controlled environments.

The device transmitted data to Firebase Realtime Database using REST-based HTTP requests. A cloud function was configured with new RTDB entries and transfer validated records to Firestore database, where aggregation and status evaluation were performed.

The evaluation focuses on the following aspects:

- Operating cycle timing
- Communication duration and Overhead
- System behavior during network interruptions
- Timestamp continuity during deep sleep cycles
- Correctness of backend aggregation

The goal of these evaluations was to ensure that the whole system architecture works reliably, even though the sensing part was simulated.

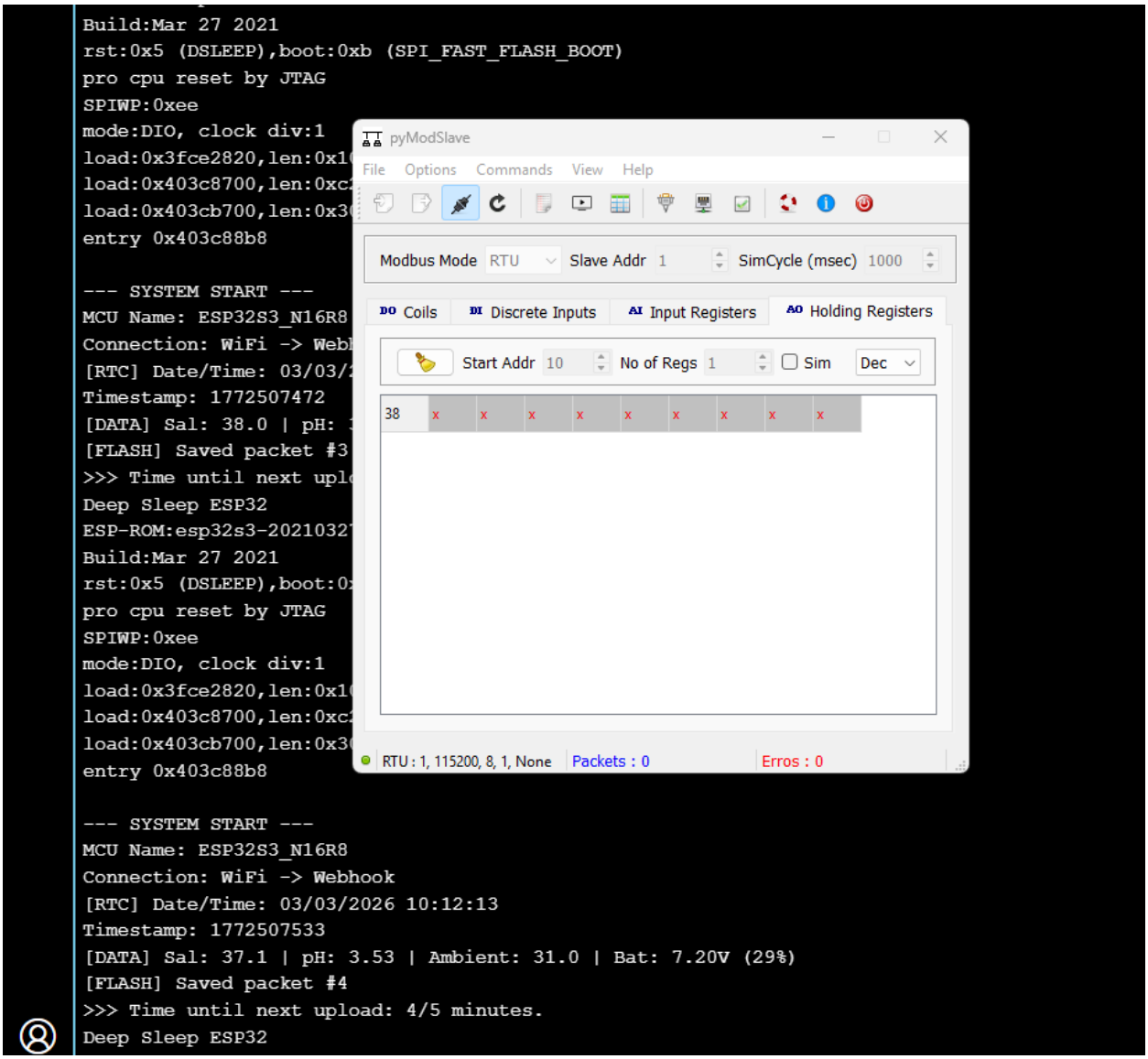

Figure 14.1 Experimental Setup

6.2 Operating Cycle and Timing Analysis

The firmware is implemented in a periodic cycle including wake-up, measurement, data handling, data transmission, and deep sleep. This section evaluates the timing behavior of one complete cycle.

When the ESP32 wakes up from deep sleep, it reinitializes required peripherals and restores necessary state variables. The initialization phase includes setting up communication interfaces and preparing for data acquisition. This stage is relatively short compared to the transmission phase.

After initialization, the firmware performs measurement acquisition. In the current implementation, values are obtained through signal emulation (ADC input or Modbus RTU). The measurement phase is brief and requires minimal processing time.

Once data is collected, it is stored locally in non-volatile memory. If the specified upload interval is met, the device activates the network interface and attempts data transmission. The transmission phase consumes most time and energy within the cycle.

After finishing all necessary tasks, the firmware configures the wake-up timer and enters deep sleep mode. During deep sleep, power consumption is significantly decreased, and the system is inactive until the next scheduled wake-up.

In general, the operating cycle can be divided into:

- Initialization phase
- Measurement phase
- Storage phase
- Transmission phase
- Deep sleep phase

Among these stages, network transmission contributes the largest portion of active time. When batch transmission is used, the number of transmission events is reduced, which lowers the overall duty cycle.

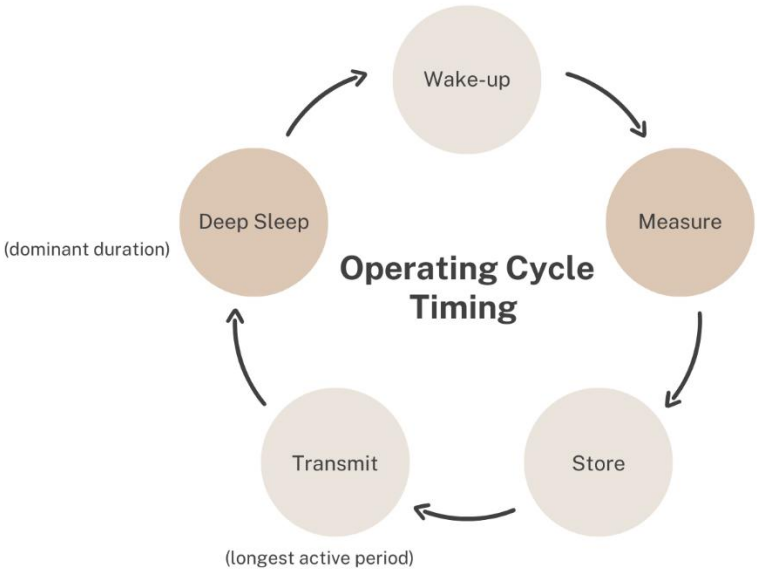

Figure 15.2 Operating Cycle Timing

6.3 Overhead Communication Evaluation

Local processing uses less energy than wireless connectivity. The firmware employs batch transmission instead of delivering each measurement one at a time for this reason.

When transmission begins after each measurement, the device must do the following repeatedly:

- Turn on Wi-Fi
- Establish a connection
- Send an HTTP request
- Wait for a response from server

It takes longer to establish a connection than to handle data locally, which increases active time.

With batch transmission, you can upload more than one record at a time. The connection process only happens once every batch, even though the payload size gets bigger.

It is clear from observation that cutting down on the number of transmissions significantly shortens the total active duration. This shows that batch transmission works better for solar-powered systems.

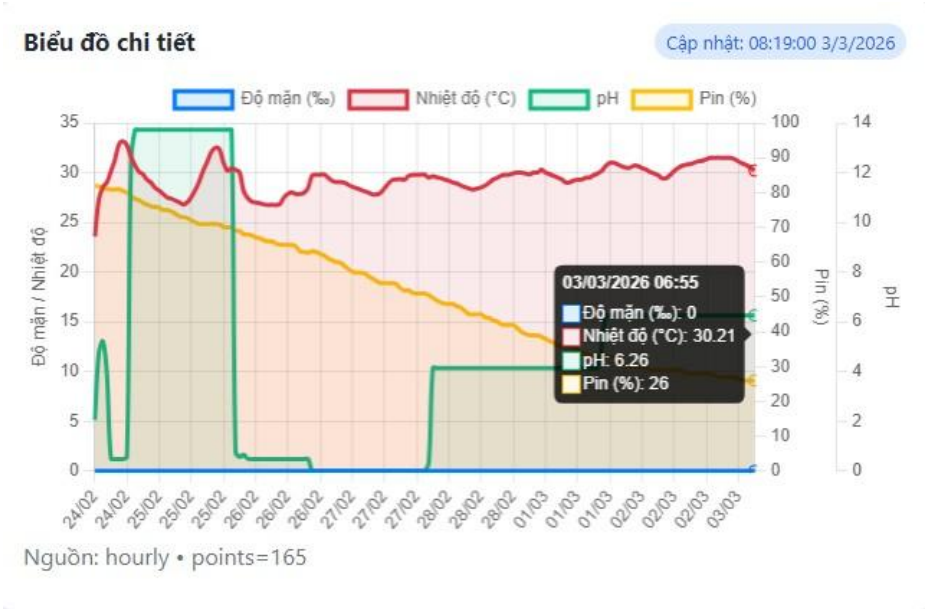

Figure 16.3.1 Estimated battery lifetime (batch upload per 5-minute)

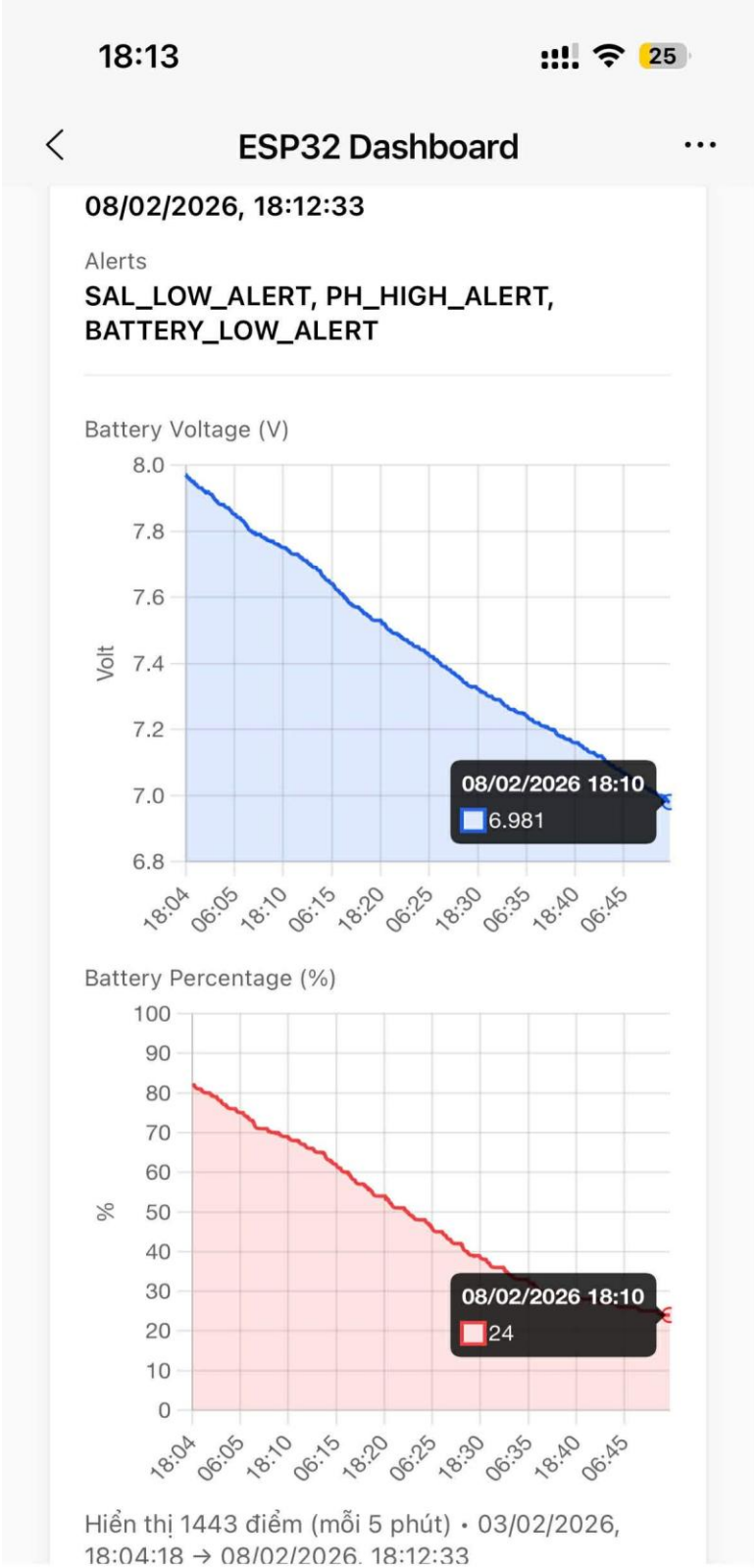

Figure 17.3.2 Estimated battery lifetime (single upload per measurement)

6.4 Data Reliability Under Network Interruption

To evaluate reliability, the system was tested under simulated network interruption.

When network connectivity was unavailable during a transmission attempt:

- The firmware did not delete buffered records.
- The system returned to deep sleep.
- Data remained stored in non-volatile storage.

After network connectivity was restored, the device uploaded previously stored measurements.

This shows that the store-and-forward system avoids data loss when communication breaks down for a short time.

The Realtime Database architecture makes data more reliable. The raw record stays in RTDB even if backend processing fails for a short time. It can be processed later.

```
load:0x403cb700,len:0x30c0
entry 0x403c88b8

--- SYSTEM START ---
MCU Name: ESP32S3_N16R8
Connection: WiFi -> Webhook
[RTC] Date/Time: 04/03/2026 02:38:07
Timestamp: 1772566687
[DATA] Sal: 38.7 | pH: 4.38 | Ambient: 31.2 | Bat: 7.12V (27%)
[FLASH] Saved packet #10
>>> Time until next upload: 5/5 minutes.
Interval reached. Connecting to WiFi...
....
WiFi Connected.
[FLASH] Found 10 offline packets. Building payload...
--- Sending Batch to Firebase via PATCH ---
HTTP Success: 200. Clearing Flash Queue.
Deep Sleep ESP32
```

Figure 18.4.1 Device upload stored measurements after network connectivity was restored

```
entry 0x403c88b8

--- SYSTEM START ---
MCU Name: ESP32S3_N16R8
Connection: WiFi -> Webhook
[RTC] Date/Time: 04/03/2026 02:32:56
Timestamp: 1772566376
[DATA] Sal: 35.7 | pH: 3.20 | Ambient: 31.2 | Bat: 7.12V (27%)
[FLASH] Saved packet #5
>>> Time until next upload: 5/5 minutes.
Interval reached. Connecting to WiFi...
.....................
>> REASON: WiFi Network Not Found (2 Blinks)
[LED] Blinking 2 times...
Deep Sleep ESP32
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x5 (DSLEEP),boot:0xb (SPI_FAST_FLASH_BOOT)
pro cpu reset by JTAG
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x10cc
load:0x403c8700,len:0xc2c
load:0x403cb700,len:0x30c0
entry 0x403c88b8

--- SYSTEM START ---
MCU Name: ESP32S3_N16R8
Connection: WiFi -> Webhook
[RTC] Date/Time: 04/03/2026 02:34:07
Timestamp: 1772566447
[DATA] Sal: 36.3 | pH: 3.25 | Ambient: 31.2 | Bat: 7.12V (27%)
[FLASH] Saved packet #6
>>> Time until next upload: 1/5 minutes.
Deep Sleep ESP32
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x5 (DSLEEP),boot:0xb (SPI_FAST_FLASH_BOOT)
pro cpu reset by JTAG
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x10cc
load:0x403c8700,len:0xc2c
load:0x403cb700,len:0x30c0
entry 0x403c88b8
```

Figure 19.4.2 Device keeps batch measurements after network interrupts

6.5 Timestamp Consistency Evaluation

For valid aggregation in the cloud, timestamps must be accurate. When the network is available, NTP is used to sync the time in this approach. After the synchronization, the firmware adds the measurement interval to the stored epoch value every time it cycles.

Testing showed that:

- Timestamps increase steadily during deep sleep cycles.
- There are no duplicate timestamps.
- The aggregation entries are updated correctly.

Even if there is a temporary network problem, timestamp progression keeps on locally until it can be re-synchronized.

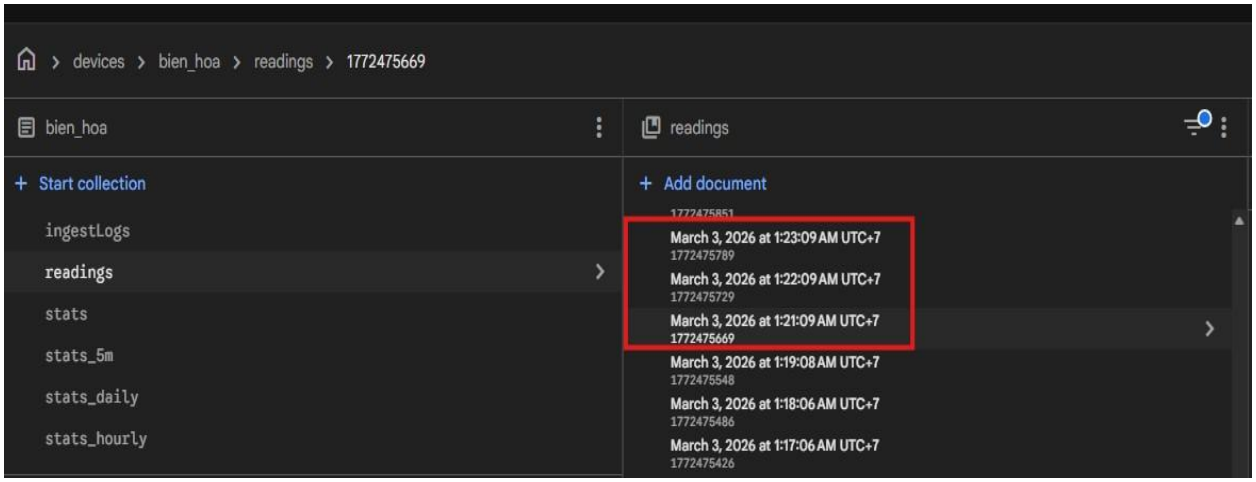

Figure 20.5.1 Sequential timestamps across multiple cycles (cloud storage)

```
--- SYSTEM START ---
MCU Name: ESP32S3_N16R8
Connection: WiFi -> Webhook
[RTC] Date/Time: 03/03/2026 01:21:09
Timestamp: 1772475669
[DATA] Sal: 38.4 | pH: 3.22 | Ambient: 26.0 | Bat: 7.22V (30%)
[FLASH] Saved packet #3
>>> Time until next upload: 3/5 minutes.
Deep Sleep ESP32
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x5 (DSLEEP),boot:0xb (SPI_FAST_FLASH_BOOT)
pro cpu reset by JTAG
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x10cc
load:0x403c8700,len:0xc2c
load:0x403cb700,len:0x30c0
entry 0x403c88b8

--- SYSTEM START ---
MCU Name: ESP32S3_N16R8
Connection: WiFi -> Webhook
[RTC] Date/Time: 03/03/2026 01:22:09
Timestamp: 1772475729
[DATA] Sal: 37.6 | pH: 3.07 | Ambient: 26.0 | Bat: 7.22V (30%)
[FLASH] Saved packet #4
>>> Time until next upload: 4/5 minutes.
Deep Sleep ESP32
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x5 (DSLEEP),boot:0xb (SPI_FAST_FLASH_BOOT)
pro cpu reset by JTAG
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2820,len:0x10cc
load:0x403c8700,len:0xc2c
load:0x403cb700,len:0x30c0
entry 0x403c88b8

--- SYSTEM START ---
MCU Name: ESP32S3_N16R8
Connection: WiFi -> Webhook
[RTC] Date/Time: 03/03/2026 01:23:09
Timestamp: 1772475789
[DATA] Sal: 39.1 | pH: 3.29 | Ambient: 26.0 | Bat: 7.22V (30%)
[FLASH] Saved packet #5
>>> Time until next upload: 5/5 minutes.
Interval reached. Connecting to WiFi...
```

Figure 21.5.2 Sequential timestamps across multiple cycles (local storage)

6.6 Backend Aggregation Verification

The final evaluation step verifies whether the backend correctly processes and aggregates uploaded data.

After data is written to RTDB and transferred to Firestore, the following were checked:

- Raw records stored correctly
- Status classification (OK/WARN/ALERT) matches threshold conditions
- Aggregation collections update properly.

For selected time intervals, manual calculations were compared against aggregated values stored in Firestore. The average, minimum, and maximum values were consistent with expected results. This proves that the aggregation logic on the cloud side works exactly.

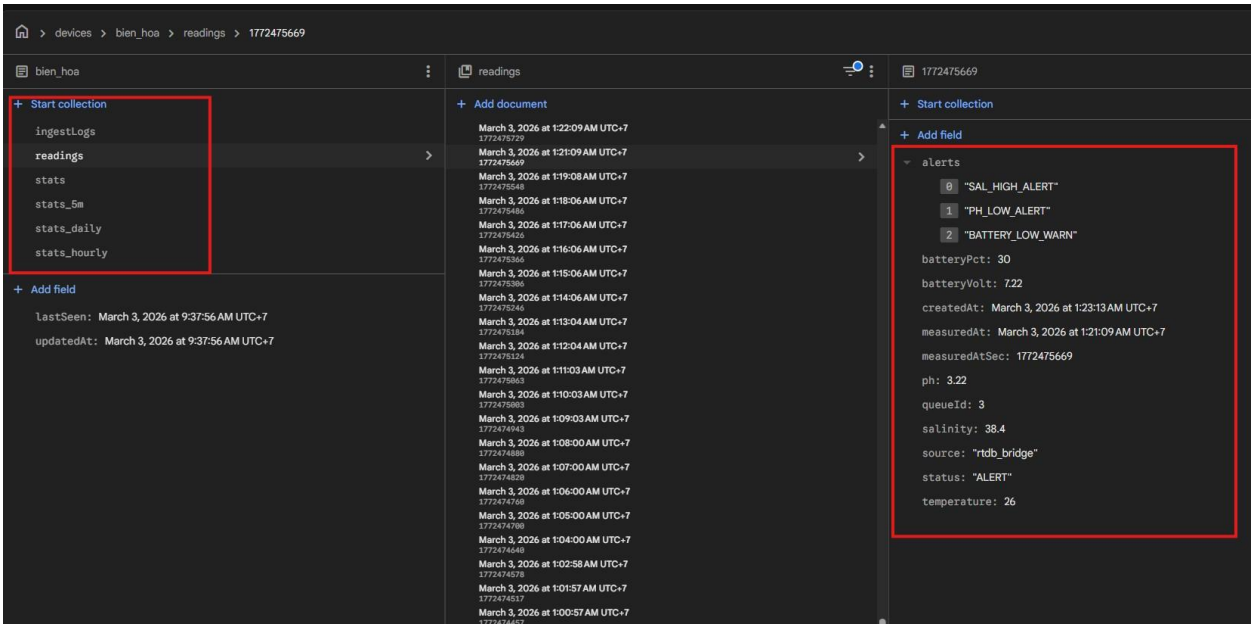

Figure 22.6 Backend Aggregation

7. Conclusion and Future work

7.1 Conclusion

This thesis presents the design and implementation of a battery-powered wireless monitoring system based on the ESP32-S3 firmware. The main work focuses on developing firmware and processing on the cloud backend.

On the embedded side, a low-power operating model was used that included deep sleep control and periodic wake-up scheduling. The firmware includes local buffering, batch transmission, and timestamp management to make sure that the device works reliably even when the network conditions change. With the store-and-forward feature, the non-volatile storage can keep measurements even when there is no internet connection and send them later without losing any data.

On the cloud side, a Realtime Database design was implemented. Firstly, the measurement data is written to Firebase Realtime Database. Then, an event-triggered cloud function process it. At different time resolutions, the backend checks for errors, sets thresholds, and combines statistics. The split between ingestion and processing makes the system easier to maintain and more open.

Experimental evaluation proved that:

- Deep sleep plays an important role in the operation cycle, and it helps the whole system save energy.
- Batch transmission cuts down on the cost of communication
- Timestamp maintains across cycles
- Cloud aggregation gives accurate statistical data

The system architecture and data flow were tested under actual scenarios, while salinity and pH values were simulated throughout development.

The proposed design shows a realistic way to build a low-power wireless monitoring node that can be scalable with cloud computing.

7.2 Limitations

Even though the system architecture has been successfully operated, there are still some problems.

Firstly, the physical salinity and pH probes were not put together throughout the development process. Signal simulation was used to provide sensor results, which do not show practical sensor noise, calibration drift, or interference from the surroundings.

Secondly, although battery voltage and percentage were monitored through the onboard management circuit, accurate current profiles were not monitored by using special power measurement equipment. Because of this, energy evaluation was based on operational behavior and timing analysis rather than precise phase-by-phase current measurements

Finally, just a small number of devices were tested. We did not test large-scale deployment scenarios in the lab, also outside realistic.

7.3 Future Work

There are several improvements that can be developed in the future. The most immediate extension is adding real salinity and pH sensors. This would allow evaluation of calibration procedures, sensor stability, and long-term drift under actual environmental conditions.

On the embedded side, future work might integrate:

- A detailed profile of current consumption
- Dynamic scheduling of transmissions dependent on battery level
- Changing the time between measurements based on changes in the environment

On the backend, other improvements could be:

- Automatic systems send alert notifications to users.
- Using past trends to make long-term trend predictions
- Classify and process with incorrect raw values.

Finally, testing with multiple deployed nodes would allow evaluation of scalability and cloud performance under higher traffic conditions

References

[1] R. L. N. e. al, "Aquaculture in the 21st century," *Proceedings of the National Academy of Sciences,* vol. 98, no. 13, p. 7270–7275, 2001.

[2] I. F. A. e. al, "Wireless sensor networks: a survey," *Computer Networks,* vol. 38, no. 4, p. 393–422, 2002.

[3] B. M. a. D. G. J. Yick, "Wireless sensor network survey," *Computer Networks,* vol. 52, no. 12, p. 2292–2330, 2008.

[4] W. H. a. S. L. L. Da Xu, "Internet of Things in industries: A survey," *IEEE Transactions on Industrial Informatics,* vol. 10, no. 4, p. 2233–2243, 2014.

[5] A. K. e. al, "Power management in energy harvesting sensor networks," *ACM Transactions on Embedded Computing Systems,* vol. 6, no. 4, 2007.

[6] M. A. e. al, "Energy conservation in wireless sensor networks: A survey," *Ad Hoc Networks,* vol. 7, no. 3, p. 537–568, 2009.

[7] L. G. a. D. E. J. Elson, "Fine-grained network time synchronization using reference broadcasts," in *OSDI*, 2002.

[8] H. A. A. A. e. al., "Wireless sensor network for water quality monitoring," *IEEE Access,* 2019.

[9] A. Y. a. J. C. M. Pule, "Wireless sensor networks: A survey on monitoring water quality," *Journal of Applied Research and Technology,* 2017.

[10] D. Evans, "The Internet of Things: How the next evolution of the Internet is changing everything," *Cisco White Paper,* 2011.

[11] J. H. S. Z. a. M. B. S. S. Kansal, "Power management in energy harvesting sensor networks," *ACM Transactions on Embedded Computing Systems,* vol. 6, no. 4, p. 32:1–32:38, 2007.

[12] C. A. a. C. Galperti, "An adaptive system for optimal solar energy harvesting in wireless sensor network nodes," *IEEE Transactions on Circuits and Systems I: Regular Papers,* vol. 55, no. 6, p. 1742–1750, 2008.

[13] C. S. S. P. a. M. B. S. V. Raghunathan, "Energy-aware wireless microsensor networks," *IEEE Signal Processing Magazine,* vol. 19, no. 2, p. 40–50, 2002.

[14] B. G. a. T. V. A. Dunkels, "Contiki — a lightweight and flexible operating system for tiny networked sensors," *Proc. IEEE Local Computer Networks (LCN),* 2004.

[15] J. H. a. D. C. J. Polastre, "Versatile low power media access for wireless sensor networks," in *Proc. ACM SenSys*, 2004.

[16] T. H. e. al., "Energy-efficient surveillance system using wireless sensor networks," in *Proc. ACM MobiSys*, 2004.

[17] J. S. S. a. F. B. R. Silva, "Opportunistic routing in wireless sensor networks," *Computer Communications,* vol. 34, no. 4, p. 465–475, 2011.

[18] F. Ö. a. Z. H. A. Dunkels, "An adaptive communication architecture for wireless sensor networks," in *Proc. ACM SenSys Workshop*, 2007.

[19] K. L. a. A. Meier, "Analyzing MAC protocols for low data-rate applications," *ACM Transactions on Sensor Networks,* vol. 7, no. 1, 2010.

[20] D. L. Mills, "Internet time synchronization: The Network Time Protocol," *IEEE Transactions on Communications,* vol. 39, no. 10, p. 1482–1493, 1991.

[21] R. F. e. al., "Hypertext Transfer Protocol — HTTP/1.1," IETF RFC 2616, 1999.

[22] H. K. a. W. Ochsenreiter, "Clock synchronization in distributed real-time systems," *IEEE Transactions on Computers,* vol. 36, no. 8, p. 933–940, 1987.

[23] A. B. a. R. Gupta, "MQTT Version 3.1.1," *OASIS Standard,* 2014.

[24] R. F. a. J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing," *IETF RFC 7230,* 2014.

[25] F. K. D. F. a. V. S. G. Kortuem, "Smart objects as building blocks for the Internet of Things," *IEEE Internet Computing,* vol. 14, no. 1, p. 44–51, 2010.

Appendices

Appendix A: Source Code Repository

A.1 Repository link

The complete implementation (ESP32-S3 firmware and cloud backend function) is provided in GitHub.

- Repository: https://github.com/dangvuminhduy1/fw.git

A.2 Repository structure

The repository is organised into two main folders:

firmware/ESP32-S3 firmware

cloud/Firebase Cloud Function (backend logic)

docs/Thesis Report

A.3 File list and purpose

Firmware:

- ESP32_S3_WiFi_Firmware.ino

Main Wi-Fi firmware used in the thesis demonstration. Sensor values are emulated using two potentiometers (ADC inputs). Data is uploaded to Firebase Realtime Database (RTDB).

- ESP32_S3_SIMCOM_Firmware.ino

Firmware variant using a SIMCOM 4G modem. Salinity is read via Modbus RTU over RS485 (pyModSlave used as emulator). pH is emulated with a potentiometer (ADC). Data is uploaded to Firebase RTDB.

- ESP32_S3_WiFi_TCP_Firmware(Prototype).cpp

Early prototype firmware using Modbus TCP to read emulated registers from pyModSlave and sending data directly to a Cloud Function endpoint. This variant was kept for reference but is not part of the final system workflow.

Cloud:

- backend_cloud_function.js

Cloud Function source code. Includes:

- an early "direct HTTP ingest" endpoint (ingestBatch, prototype path), and
- the main RTDB → Firestore bridge used in the final data pipeline.

Appendix B: Firmware Overview and Configuration

B.1 Common operating cycle

Both main firmware variants (Wi-Fi and SIMCOM) follow the same sampling and transmission strategy:

- Sampling: one measurement cycle every 1 minute (deep sleep timer wake-up).
- Batch upload: data is buffered locally and transmitted every 5 minutes.
- Local buffering: up to 300 records are stored (approx. 5 hours at 1-minute sampling).
- Timekeeping: DS3231 RTC is used for timestamps. Network time is used to correct RTC when available.
- Power: ESP32 enters deep sleep after each cycle; network interfaces are only enabled during the upload window.

B.2 Pin mapping (main variants)

Function	ESP32-S3 Wi-Fi firmware	ESP32-S3 SIMCOM firmware
Battery ADC input	GPIO10	GPIO10
Battery ADC enable	GPIO9	GPIO9
Debug LED	GPIO13	GPIO13

Salinity input	GPIO6 (potentiometer)	Modbus RTU via RS485
pH input	GPIO7 (potentiometer)	GPIO8 (potentiometer)
RS485 RX / TX	–	GPIO11 / GPIO12
SIMCOM RX / TX	–	GPIO4 / GPIO5
I2C SDA / SCL (DS3231)	GPIO1 / GPIO2	GPIO1 / GPIO2

Table B.2 Pin mapping of the main firmware variants

B.3 Data acquisition

- Salinity and pH (emulated)
  - Wi-Fi firmware: two potentiometers are read via ADC (salinity and pH).
  - SIMCOM firmware: pH is read via ADC potentiometer; salinity is read from Modbus RTU holding register(s).
- Temperature
  - DS3231 internal temperature sensor is used as the ambient temperature signal in both variants.
- Battery
  - Battery voltage is read via ADC with an enable pin to reduce measurement power draw.
  - Battery percentage is derived from the measured voltage using a piece-wise mapping for a 2-cell battery range (approximately 6.0–8.4 V).

B.4 Local buffering format (Wi-Fi and SIMCOM)

Records are stored in non-volatile storage using ESP32 Preferences (NVS). Each record contains:

- ts (Unix timestamp in seconds)
- sal (salinity)
- ph (pH)
- amb_temp (ambient temperature)
- v (battery voltage)

- p (battery percentage)

A counter (count) is maintained to track how many records are currently queued.

B.5 Wi-Fi upload method

During the 5-minute upload window:

- Wi-Fi is enabled and connected to the configured SSID.
- All queued records are converted into a JSON object where each key is the Unix timestamp.
- A single HTTP PATCH request is sent to Firebase RTDB at /data.json.
- On success, the local queue counter is cleared. On failure, data remains for retry in the next upload window.
- Wi-Fi is disconnected and powered down before deep sleep.

B.6 SIMCOM upload method

During the 5-minute upload window

- The modem is woken up and attached to the cellular network using the configured APN.
- Queued records are converted to a JSON object (timestamp-keyed).
- Data is uploaded to Firebase RTDB using SIMCOM HTTP AT commands with SSL enabled.
- The request URL uses an HTTP method override to ensure “PATCH-like” merge behaviour on RTDB
- After completion, the modem is put back to low-power mode and the ESP32 enters deep sleep.

B.7 Wi-Fi TCP (Prototype)

The prototype firmware differs from the final design in the following points:

- Uses Modbus TCP (server IP and port configured in the source) to read registers from pyModSlave.
- Buffers data in LittleFS using a text file and a small metadata file.

- Sends records directly to a Cloud Function endpoint (ingestBatch) using HTTP POST with a device key.
- This path was replaced by the RTDB → Firestore bridge approach, but the prototype remains in the repository for traceability.

Appendix C: Sensor Emulation and Test Setup

C.1 Potentiometer-based emulation (Wi-Fi firmware)

Two potentiometers are used to emulate salinity and pH signals:

- Each potentiometer is wired as a voltage divider between 3.3 V and GND.
- The wiper output is connected to:
    - salinity ADC pin (GPIO6), and
    - pH ADC pin (GPIO7).

In the firmware, the ADC voltage range is mapped to representative measurement ranges:

- Salinity: 0–100 (representative ppt scale)
- pH: 0.00–14.00

C.2 Modbus RTU emulation with pyModSlave (SIMCOM firmware)

Salinity is emulated using pyModSlave in Modbus RTU mode

- Slave (Unit ID): 1
- Register type: Holding Register
- Salinity source register: address 0x00 (one register)

The ESP32 communicates through an RS485 transceiver (UART + RS485). The returned register value is used as the salinity reading in the firmware.

C.3 Modbus TCP emulation with pyModSlave (prototype firmware)

The prototype reads three holding registers over Modbus TCP

- Register 0: salinity

- Register 1: temperature
- Register 2: pH

This setup was used only during early development and is not required for the final system pipeline.

Appendix D: Cloud Backend Functions and Data Model

D.1 Location

Cloud backend source is located at:

- cloud/backend_cloud_function.js

D.2 Two ingestion paths in the Cloud Function

1. **Prototype path (HTTP ingest)**

An HTTP endpoint receives batched JSON records from devices and writes directly to Firestore. This approach was used early in development and later replaced.

2. **Main path (RTDB → Firestore bridge)**

A database trigger listens for new entries created under RTDB /data/{id} and transfers the record to Firestore. This is the approach used for the final system workflow.

D.3 RTDB record format (main path)

Each RTDB child under /data/ contains:

- device_id
- salinity
- ph
- ambient_temp
- bat_p (battery percentage)
- bat_v (battery voltage)
- queue_id

- measuredAtSec (Unix time in seconds)

D.4 Firestore structure (main path)

Firestore is organised per device:

- devices/{deviceId}
- readings/{id}: one document per RTDB entry
- stats/latest: last measurement snapshot
- stats_5m/{bucketKey}: 5-minute aggregation
- stats_hourly/{bucketKey}: hourly aggregation
- stats_daily/{bucketKey}: daily aggregation
- ingestLogs/{autoId}: logs for ingestion activity

D.5 Status classification and alert codes

Each reading is classified into one of three states:

- OK
- WARN
- ALERT

The classification is generated using threshold ranges for:

- salinity
- temperature
- pH
- battery percentage

Alert codes are stored as a list (e.g., low/high warn and alert codes per parameter).

D.6 Aggregation windows

Statistics are calculated for:

- 5-minute buckets
- hourly buckets

- daily buckets

Bucket keys are generated using local time (UTC+7) for consistent grouping, while timestamps are still stored as Unix epoch seconds

For each bucket, Firestore stores:

- count
- sum/min/max per parameter
- average values (rounded)
- bucketStartSec and a Firestore Timestamp representation
- updatedAt

D.7 Migration tool

A utility endpoint exists to migrate older RTDB records into Firestore in batches. The endpoint supports parameters such as:

- limit
- startAfter

Appendix E: Sample JSON Payloads and Database Structure

E.1 Example Wi-Fi/SIMCOM payload written to RTDB (PATCH body)

Example structure of a batch payload (timestamp keys):

```
{
  "1712140800": {
    "device_id": "HoChiMinh_city",
    "queue_id": 1,
    "measuredAtSec": 1712140800,
    "salinity": 2.4,
    "ph": 7.10,
    "ambient_temp": 26.5,
    "bat_v": 7.80,
    "bat_p": 82
  },
  "1712140860": {
```

```
    "device_id": "HoChiMinh_city",
    "queue_id": 2,
    "measuredAtSec": 1712140860,
    "salinity": 2.5,
    "ph": 7.05,
    "ambient_temp": 26.6,
    "bat_v": 7.79,
    "bat_p": 82
  }
}
```

E.2 Example prototype request body (HTTP ingestBatch path)

```
{
  "deviceId": "demo_wifi_1",
  "key": "abc123",
  "records": [
    {
      "sal": 2.4,
      "temp": 26.5,
      "ph": 7.10,
      "rawSal": 2400,
      "rawTemp": 2650,
      "rawPh": 710,
      "battPct": 82,
      "battVolt": 7.80,
      "measuredAt": 1712140800
    }
  ]
}
```

E.3 Example Firestore readings/{id} document (main path)

A reading document contains:

```
{
  "device_id": "HoChiMinh_city",
  "measuredAtSec": 1741008000,
  "measuredAt": "Timestamp(2025-03-03T00:00:00Z)",
  "salinity": 2.3,
  "ph": 7.1,
  "temperature": 28.5,
  "batteryPct": 68,
```

```
  "batteryVolt": 7.62,
  "status": "OK",
  "alerts": [],
  "source":"rtdb_bridge",
  "createdAt": "serverTimestamp()",
  "updatedAt": "serverTimestamp()"
}
```

E.4 Example Firestore aggregation document (stats buckets)

Aggregation documents contain:

- count
- sum*, min*, max*, avg*
- bucketStartSec, bucketStart
- updatedAt